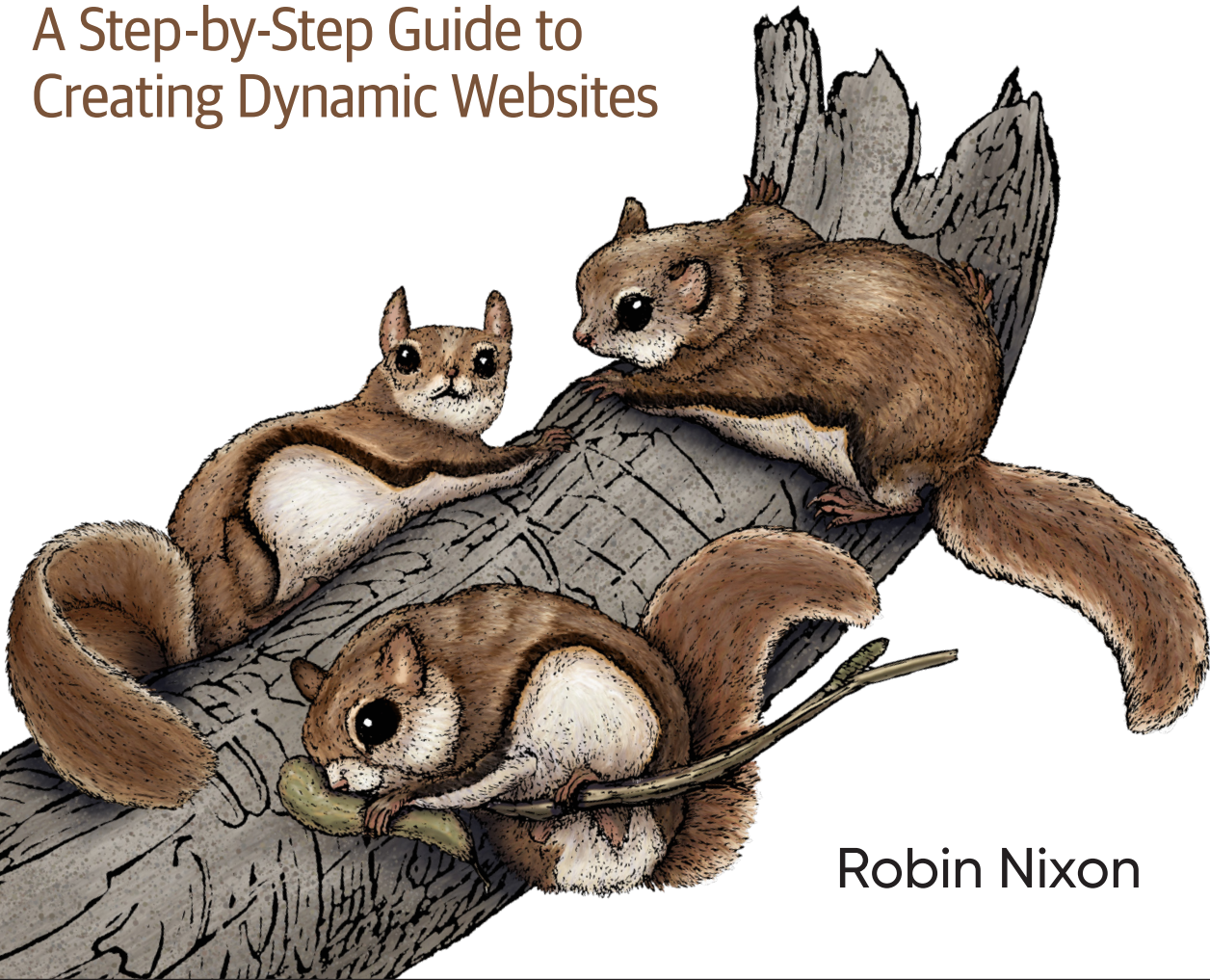


O'REILLY®

6th Edition
Now including React,
PHP 8 & MySQL 8

Learning PHP, MySQL & JavaScript

A Step-by-Step Guide to
Creating Dynamic Websites



Robin Nixon

Learning PHP, MySQL & JavaScript

Build interactive, data-driven websites with the potent combination of open source technologies and web standards, even if you have only basic HTML knowledge. With the latest edition of this popular hands-on guide, you'll tackle dynamic web programming using the most recent versions of today's core technologies: PHP, MySQL, JavaScript, CSS, HTML5, jQuery, and the powerful React library.

Web designers will learn how to use these technologies together while picking up valuable web programming practices along the way—including how to optimize websites for mobile devices. You'll put everything together to build a fully functional social networking site suitable for both desktop and mobile browsers.

- Explore MySQL from database structure to complex queries
- Use the MySQL PDO extension, PHP's improved MySQL interface
- Create dynamic PHP web pages that tailor themselves to the user
- Manage cookies and sessions and maintain a high level of security
- Enhance JavaScript with the React library
- Use Ajax calls for background browser-server communication
- Style your web pages by acquiring CSS skills
- Implement HTML5 features, including geolocation, audio, video, and the canvas element
- Reformat your websites into mobile web apps

"This is a great beginner's book that introduces several crucial web developer languages. It's a quick-paced, easy-to-read, information-packed book that will soon have you creating dynamically driven websites, including a basic social networking site."

—Albert Wiersch

Developer of CSE HTML Validator

Robin Nixon has worked with and written about computers since the early 1980s. One of the websites he developed presented the world's first radio station licensed by the music copyright holders. He also developed the first known pop-up windows to enable people to surf while listening. Robin also worked full time for one of Britain's main IT magazine publishers, where he held several roles including editorial, promotions, and cover disc editing.

WEB PLATFORM

US 59.99

CAN 79.99

ISBN: 978-1-492-09382-4



Twitter: @oreillymedia
facebook.com/oreilly

SIXTH EDITION

Learning PHP, MySQL & JavaScript

*A Step-by-Step Guide
to Creating Dynamic Websites*

Robin Nixon

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Learning PHP, MySQL & JavaScript

by Robin Nixon

Copyright © 2021 Robin Nixon. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Amanda Quinn

Development Editor: Melissa Potter

Production Editor: Caitlin Ghegan

Copyeditor: Kim Cofer

Proofreader: Judith McConville

Indexer: Kim Sandoval

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

July 2021: Sixth Edition

Revision History for the Sixth Edition

2021-07-21: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492093824> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Learning PHP, MySQL & JavaScript*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors, and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-492-09382-4

[LSI]

For Julie, Naomi, Harry, Matthew, Laura, Hannah, Rachel, and David

Table of Contents

Preface.....	xxiii
1. Introduction to Dynamic Web Content.....	1
HTTP and HTML: Berners-Lee's Basics	2
The Request/Response Procedure	2
The Benefits of PHP, MySQL, JavaScript, CSS, and HTML5	5
MariaDB: The MySQL Clone	6
Using PHP	6
Using MySQL	7
Using JavaScript	8
Using CSS	10
And Then There's HTML5	10
The Apache Web Server	11
Handling Mobile Devices	12
About Open Source	12
Bringing It All Together	13
Questions	15
2. Setting Up a Development Server.....	17
What Is a WAMP, MAMP, or LAMP?	18
Installing AMPPS on Windows	18
Testing the Installation	23
Accessing the Document Root (Windows)	25
Alternative WAMPs	26
Installing AMPPS on macOS	27
Accessing the Document Root (macOS)	28
Installing a LAMP on Linux	29
Working Remotely	29

Logging In	30
Using SFTP or FTPS	30
Using a Code Editor	31
Using an IDE	32
Questions	34
3. Introduction to PHP.....	35
Incorporating PHP Within HTML	35
This Book's Examples	37
The Structure of PHP	37
Using Comments	37
Basic Syntax	38
Variables	39
Operators	44
Variable Assignment	48
Multiline Commands	50
Variable Typing	52
Constants	53
Predefined Constants	54
The Difference Between the echo and print Commands	55
Functions	55
Variable Scope	56
Questions	62
4. Expressions and Control Flow in PHP.....	63
Expressions	63
TRUE or FALSE?	64
Literals and Variables	65
Operators	66
Operator Precedence	67
Associativity	69
Relational Operators	70
Conditionals	74
The if Statement	75
The else Statement	76
The elseif Statement	78
The switch Statement	79
The ? (or ternary) Operator	82
Looping	83
while Loops	84
do...while Loops	86
for Loops	86

Breaking Out of a Loop	88
The continue Statement	89
Implicit and Explicit Casting	90
PHP Dynamic Linking	91
Dynamic Linking in Action	92
Questions	93
5. PHP Functions and Objects.....	95
PHP Functions	96
Defining a Function	97
Returning a Value	98
Returning an Array	99
Passing Arguments by Reference	100
Returning Global Variables	102
Recap of Variable Scope	102
Including and Requiring Files	103
The include Statement	103
Using include_once	103
Using require and require_once	104
PHP Version Compatibility	104
PHP Objects	105
Terminology	106
Declaring a Class	107
Creating an Object	108
Accessing Objects	108
Cloning Objects	110
Constructors	111
Destructors	111
Writing Methods	112
Declaring Properties	113
Declaring Constants	114
Property and Method Scope	114
Static Methods	115
Static Properties	116
Inheritance	117
Questions	121
6. PHP Arrays.....	123
Basic Access	123
Numerically Indexed Arrays	123
Associative Arrays	125
Assignment Using the array Keyword	126

The foreach...as Loop	127
Multidimensional Arrays	129
Using Array Functions	132
is_array	132
count	133
sort	133
shuffle	133
explode	134
extract	135
compact	135
reset	137
end	137
Questions	137
7. Practical PHP.....	139
Using printf	139
Precision Setting	140
String Padding	142
Using sprintf	143
Date and Time Functions	143
Date Constants	146
Using checkdate	146
File Handling	147
Checking Whether a File Exists	147
Creating a File	147
Reading from Files	149
Copying Files	150
Moving a File	150
Deleting a File	151
Updating Files	151
Locking Files for Multiple Accesses	152
Reading an Entire File	154
Uploading Files	155
System Calls	160
XHTML or HTML5?	162
Questions	163
8. Introduction to MySQL.....	165
MySQL Basics	165
Summary of Database Terms	166
Accessing MySQL via the Command Line	166
Starting the Command-Line Interface	167

Using the Command-Line Interface	171
MySQL Commands	172
Data Types	177
Indexes	187
Creating an Index	187
Querying a MySQL Database	193
Joining Tables Together	203
Using Logical Operators	205
MySQL Functions	206
Accessing MySQL via phpMyAdmin	206
Questions	207
9. Mastering MySQL.....	209
Database Design	209
Primary Keys: The Keys to Relational Databases	210
Normalization	211
First Normal Form	212
Second Normal Form	214
Third Normal Form	216
When Not to Use Normalization	218
Relationships	219
One-to-One	219
One-to-Many	220
Many-to-Many	221
Databases and Anonymity	222
Transactions	223
Transaction Storage Engines	223
Using BEGIN	224
Using COMMIT	225
Using ROLLBACK	225
Using EXPLAIN	226
Backing Up and Restoring	227
Using mysqldump	227
Creating a Backup File	229
Restoring from a Backup File	231
Dumping Data in CSV Format	231
Planning Your Backups	232
Questions	232
10. What's New in PHP 8 and MySQL 8.....	235
About This Chapter	235
PHP 8	236

Named Parameters	236
Attributes	237
Constructor Properties	237
Just-In-Time Compilation	238
Union Types	238
Null-safe Operator	238
match Expressions	238
New Functions	239
MySQL 8	243
Updates to SQL	243
JSON (JavaScript Object Notation)	244
Geography Support	244
Reliability	244
Speed and Performance	244
Management	245
Security	245
Questions	246
11. Accessing MySQL Using PHP.....	247
Querying a MySQL Database with PHP	247
The Process	247
Creating a Login File	248
Connecting to a MySQL Database	249
A Practical Example	254
The \$_POST Array	257
Deleting a Record	258
Displaying the Form	258
Querying the Database	259
Running the Program	260
Practical MySQL	261
Creating a Table	262
Describing a Table	263
Dropping a Table	264
Adding Data	264
Retrieving Data	265
Updating Data	266
Deleting Data	267
Using AUTO_INCREMENT	267
Performing Additional Queries	269
Preventing Hacking Attempts	270
Steps You Can Take	271
Using Placeholders	273

Preventing JavaScript Injection into HTML	275
Questions	277
12. Form Handling.....	279
Building Forms	279
Retrieving Submitted Data	281
Default Values	282
Input Types	283
Sanitizing Input	290
An Example Program	292
HTML5 Enhancements	294
The autocomplete Attribute	295
The autofocus Attribute	295
The placeholder Attribute	295
The required Attribute	296
Override Attributes	296
The width and height Attributes	296
The min and max Attributes	296
The step Attribute	296
The form Attribute	297
The list Attribute	297
The color Input Type	297
The number and range Input Types	297
Date and Time Pickers	298
Questions	298
13. Cookies, Sessions, and Authentication.....	299
Using Cookies in PHP	299
Setting a Cookie	301
Accessing a Cookie	302
Destroying a Cookie	302
HTTP Authentication	302
Storing Usernames and Passwords	306
An Example Program	308
Using Sessions	311
Starting a Session	312
Ending a Session	315
Setting a Timeout	316
Session Security	317
Questions	320

14. Exploring JavaScript.....	321
JavaScript and HTML Text	322
Using Scripts Within a Document Head	323
Older and Nonstandard Browsers	324
Including JavaScript Files	324
Debugging JavaScript Errors	325
Using Comments	325
Semicolons	326
Variables	326
String Variables	327
Numeric Variables	327
Arrays	327
Operators	328
Arithmetic Operators	328
Assignment Operators	329
Comparison Operators	329
Logical Operators	330
Incrementing, Decrementing, and Shorthand Assignment	330
String Concatenation	330
Escape Characters	331
Variable Typing	331
Functions	333
Global Variables	333
Local Variables	333
Using let and const	334
The Document Object Model	337
Another Use for the \$ Symbol	339
Using the DOM	340
About document.write	341
Using console.log	341
Using alert	341
Writing into Elements	341
Using document.write	341
Questions	342
15. Expressions and Control Flow in JavaScript.....	343
Expressions	343
Literals and Variables	344
Operators	345
Operator Precedence	346
Associativity	346
Relational Operators	347

The with Statement	350
Using onerror	351
Using try...catch	352
Conditionals	353
The if Statement	353
The else Statement	353
The switch Statement	354
The ? Operator	356
Looping	357
while Loops	357
do...while Loops	357
for Loops	358
Breaking Out of a Loop	359
The continue Statement	359
Explicit Casting	360
Questions	361
16. JavaScript Functions, Objects, and Arrays.	363
JavaScript Functions	363
Defining a Function	363
Returning a Value	365
Returning an Array	367
JavaScript Objects	368
Declaring a Class	368
Creating an Object	370
Accessing Objects	370
The prototype Keyword	370
JavaScript Arrays	373
Numeric Arrays	374
Associative Arrays	375
Multidimensional Arrays	376
Using Array Methods	377
Questions	382
17. JavaScript and PHP Validation and Error Handling.	385
Validating User Input with JavaScript	385
The validate.html Document (Part 1)	386
The validate.html Document (Part 2)	388
Regular Expressions	391
Matching Through Metacharacters	392
Fuzzy Character Matching	392
Grouping Through Parentheses	393

Character Classes	394
Indicating a Range	394
Negation	394
Some More Complicated Examples	395
Summary of Metacharacters	398
General Modifiers	399
Using Regular Expressions in JavaScript	400
Using Regular Expressions in PHP	400
Redisplaying a Form After PHP Validation	401
Questions	407
18. Using Asynchronous Communication.....	409
What Is Asynchronous Communication?	410
Using XMLHttpRequest	410
Your First Asynchronous Program	411
Using GET Instead of POST	415
Sending XML Requests	417
Using Frameworks for Asynchronous Communication	421
Questions	421
19. Introduction to CSS.....	423
Importing a Stylesheet	424
Importing CSS from Within HTML	424
Embedded Style Settings	425
Using IDs	425
Using Classes	425
Using Semicolons	426
CSS Rules	426
Multiple Assignments	426
Using Comments	427
Style Types	428
Default Styles	428
User Styles	428
External Stylesheets	429
Internal Styles	429
Inline Styles	430
CSS Selectors	430
The Type Selector	430
The Descendant Selector	430
The Child Selector	431
The ID Selector	432
The Class Selector	433

The Attribute Selector	434
The Universal Selector	434
Selecting by Group	435
The CSS Cascade	436
Stylesheet Creators	436
Stylesheet Methods	436
Stylesheet Selectors	437
The Difference Between div and span Elements	439
Measurements	441
Fonts and Typography	443
font-family	443
font-style	444
font-size	444
font-weight	445
Managing Text Styles	445
Decoration	446
Spacing	446
Alignment	447
Transformation	447
Indenting	447
CSS Colors	448
Short Color Strings	448
Gradients	449
Positioning Elements	450
Absolute Positioning	450
Relative Positioning	451
Fixed Positioning	451
Pseudoclasses	453
Shorthand Rules	455
The Box Model and Layout	456
Setting Margins	456
Applying Borders	458
Adjusting Padding	459
Object Contents	461
Questions	461
20. Advanced CSS with CSS3.....	463
Attribute Selectors	464
Matching Parts of Strings	464
The box-sizing Property	465
CSS3 Backgrounds	466
The background-clip Property	466

The background-origin Property	468
The background-size Property	468
Using the auto Value	469
Multiple Backgrounds	469
CSS3 Borders	471
The border-color Property	471
The border-radius Property	471
Box Shadows	474
Element Overflow	475
Multicolumn Layout	475
Colors and Opacity	477
HSL Colors	477
HSLA Colors	478
RGB Colors	478
RGBA Colors	479
The opacity Property	479
Text Effects	479
The text-shadow Property	479
The text-overflow Property	479
The word-wrap Property	480
Web Fonts	481
Google Web Fonts	482
Transformations	483
3D Transformations	484
Transitions	485
Properties to Transition	485
Transition Duration	486
Transition Delay	486
Transition Timing	486
Shorthand Syntax	487
Questions	488
21. Accessing CSS from JavaScript.....	491
Revisiting the getElementById Function	491
The O Function	491
The S Function	492
The C Function	493
Including the Functions	494
Accessing CSS Properties from JavaScript	494
Some Common Properties	495
Other Properties	496
Inline JavaScript	498

The this Keyword	498
Attaching Events to Objects in a Script	499
Attaching to Other Events	499
Adding New Elements	500
Removing Elements	502
Alternatives to Adding and Removing Elements	502
Using Interrupts	503
Using setTimeout	504
Canceling a Timeout	505
Using setInterval	505
Using Interrupts for Animation	507
Questions	509
22. Introduction to jQuery.....	511
Why jQuery?	512
Including jQuery	512
Choosing the Right Version	513
Downloading	514
Using a Content Delivery Network	514
Customizing jQuery	515
jQuery Syntax	515
A Simple Example	516
Avoiding Library Conflicts	517
Selectors	517
The css Method	517
The Element Selector	518
The ID Selector	519
The Class Selector	519
Combining Selectors	519
Handling Events	520
Waiting Until the Document Is Ready	521
Event Functions and Properties	522
The blur and focus Events	523
The this Keyword	524
The click and dblclick Events	524
The keypress Event	525
Considerate Programming	527
The mousemove Event	527
Other Mouse Events	530
Alternative Mouse Methods	531
The submit Event	532
Special Effects	533

Hiding and Showing	534
The toggle Method	535
Fading In and Out	536
Sliding Elements Up and Down	537
Animations	538
Stopping Animations	541
Manipulating the DOM	542
The Difference Between the text and html Methods	543
The val and attr Methods	543
Adding and Removing Elements	545
Dynamically Applying Classes	547
Modifying Dimensions	547
The width and height Methods	548
The innerWidth and innerHeight Methods	550
The outerWidth and outerHeight Methods	550
DOM Traversal	551
Parent Elements	551
Child Elements	555
Sibling Elements	555
Selecting the Next and Previous Elements	557
Traversing jQuery Selections	559
The is Method	560
Using jQuery Without Selectors	562
The \$.each Method	562
The \$.map Method	563
Using Asynchronous Communication	563
Using the POST Method	564
Using the GET Method	564
Plug-ins	565
jQuery User Interface	565
Other Plug-ins	566
Questions	566
23. Introduction to jQuery Mobile.....	569
Including jQuery Mobile	570
Getting Started	571
Linking Pages	573
Linking Synchronously	574
Linking Within a Multipage Document	574
Page Transitions	575
Styling Buttons	580
List Handling	582

Filterable Lists	583
List Dividers	585
What Next?	588
Questions	588
24. Introduction to React.	591
What Is the Point of React Anyway?	592
Accessing the React Files	593
Including babel.js	594
Our First React Project	595
Using a Function Instead of a Class	596
Pure and Impure Code: A Golden Rule	598
Using Both a Class and a Function	598
Props and Components	599
The Differences Between Using a Class and a Function	600
React State and Life Cycle	601
Using Hooks (if You Use Node.js)	604
Events in React	604
Inline JSX Conditional Statements	606
Using Lists and Keys	607
Unique Keys	608
Handling Forms	609
Using Text Input	610
Using textarea	612
Using select	613
React Native	615
Creating React Native Apps	615
Further Reading	615
Taking React to the Next Level	616
Questions	616
25. Introduction to HTML5.	619
The Canvas	619
Geolocation	621
Audio and Video	623
Forms	624
Local Storage	624
Web Workers	624
Questions	625
26. The HTML5 Canvas.	627
Creating and Accessing a Canvas	627

The toDataURL Function	629
Specifying an Image Type	631
The fillRect Method	631
The clearRect Method	631
The strokeRect Method	632
Combining These Commands	632
The createLinearGradient Method	633
The addColorStop Method in Detail	635
The createRadialGradient Method	636
Using Patterns for Fills	638
Writing Text to the Canvas	640
The strokeText Method	640
The textBaseline Property	641
The font Property	641
The textAlign Property	641
The fillText Method	642
The measureText Method	643
Drawing Lines	643
The lineWidth Property	644
The lineCap and lineJoin Properties	644
The miterLimit Property	646
Using Paths	646
The moveTo and lineTo Methods	647
The stroke Method	647
The rect Method	648
Filling Areas	648
The clip Method	650
The isPointInPath Method	653
Working with Curves	654
The arc Method	654
The arcTo Method	657
The quadraticCurveTo Method	658
The bezierCurveTo Method	659
Manipulating Images	660
The drawImage Method	660
Resizing an Image	661
Selecting an Image Area	661
Copying from a Canvas	663
Adding Shadows	663
Editing at the Pixel Level	665
The getImageData Method	665
The putImageData Method	668

The createImageData Method	668
Advanced Graphical Effects	669
The globalCompositeOperation Property	669
The globalAlpha Property	672
Transformations	672
The scale Method	673
The save and restore Methods	674
The rotate Method	674
The translate Method	675
The transform Method	676
The setTransform Method	678
Questions	679
27. HTML5 Audio and Video.....	681
About Codecs	682
The <audio> Element	683
The <video> Element	685
The Video Codecs	686
Questions	689
28. Other HTML5 Features.....	691
Geolocation and the GPS Service	691
Other Location Methods	692
Geolocation and HTML5	693
Local Storage	696
Using Local Storage	696
The localStorage Object	697
Web Workers	699
Drag and Drop	702
Cross-Document Messaging	704
Other HTML5 Tags	708
Questions	708
29. Bringing It All Together.....	709
Designing a Social Networking App	710
On the Website	710
functions.php	710
The Functions	711
header.php	713
setup.php	716
index.php	718
signup.php	719

Checking for Username Availability	719
Logging In	720
checkuser.php	723
login.php	724
profile.php	726
Adding the “About Me” Text	727
Adding a Profile Image	727
Processing the Image	727
Displaying the Current Profile	728
members.php	731
Viewing a User’s Profile	732
Adding and Dropping Friends	732
Listing All Members	732
friends.php	735
messages.php	738
logout.php	741
styles.css	743
javascript.js	745
Questions	746
A. Solutions to the Chapter Questions.....	747
Index.....	771

Preface

The combination of PHP and MySQL is the most convenient approach to dynamic, database-driven web design, holding its own in the face of challenges from some other integrated frameworks that are harder to learn. Due to its open source roots, it is free to implement and is therefore an extremely popular option for web development.

Any would-be developer on a Unix/Linux or even a Windows platform will need to master these technologies. And, combined with the partner technologies of JavaScript, React, CSS, and HTML5, you will be able to create websites of the caliber of industry standards like Facebook, Twitter, and Gmail.

Audience

This book is for people who wish to learn how to create effective and dynamic websites. This may include webmasters or graphic designers who have already mastered creating static websites, or a CMS such as WordPress, but wish to take their skills to the next level, as well as high school and college students, recent graduates, and self-taught individuals.

In fact, anyone ready to learn the fundamentals behind responsive web design will obtain a thorough grounding in the core technologies of PHP, MySQL, JavaScript, CSS, and HTML5, and you'll learn the basics of the React library, too.

Assumptions This Book Makes

This book assumes that you have a basic understanding of HTML and can at least put together a simple, static website but does not assume that you have any prior knowledge of PHP, MySQL, JavaScript, CSS, or HTML5—although if you do, your progress through the book will be even quicker.

Organization of This Book

The chapters in this book are written in a specific order, first introducing all of the core technologies it covers and then walking you through their installation on a web development server so that you will be ready to work through the examples.

In the first section, you will gain a grounding in the PHP programming language, covering the basics of syntax, arrays, functions, and object-oriented programming.

Then, with PHP under your belt, you will move on to an introduction to the MySQL database system, where you will learn everything from how MySQL databases are structured to how to generate complex queries.

After that, you will learn how you can combine PHP and MySQL to start creating your own dynamic web pages by integrating forms and other HTML features. You will then get down to the nitty-gritty practical aspects of PHP and MySQL development by learning a variety of useful functions and how to manage cookies and sessions, as well as how to maintain a high level of security.

In the next few chapters, you will gain a thorough grounding in JavaScript, from simple functions and event handling to accessing the Document Object Model, in-browser validation, and error handling. You'll also get a comprehensive primer on using the popular React library for JavaScript.

With an understanding of all three of these core technologies, you will then learn how to make behind-the-scenes Ajax calls and turn your websites into highly dynamic environments.

Next, you'll spend two chapters learning all about using CSS to style and lay out your web pages, before discovering how the React libraries can make your development job a great deal easier. You'll then move on to the final section on the interactive features built into HTML5, including geolocation, audio, video, and the canvas. After this, you'll put together everything you've learned in a complete set of programs that together constitute a fully functional social networking website.

Along the way, you'll find plenty of advice on good programming practices and tips that can help you find and solve hard-to-detect programming errors. There are also plenty of links to websites containing further details on the topics covered.

Conventions Used in This Book

The following typographical conventions are used in this book:

Plain text

Indicates menu titles, options, and buttons.

Italic

Indicates new terms, URLs, email addresses, filenames, file extensions, pathnames, directories, and Unix utilities. Also used for database, table, and column names.

Constant width

Indicates commands and command-line options, variables and other code elements, HTML tags, and the contents of files.

Constant width bold

Shows program output and is used to highlight sections of code that are discussed in the text.

Constant width italic

Shows text that should be replaced with user-supplied values.



This element signifies a general note.



This element indicates a warning or caution.



This element indicates a tip or general note.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download on [GitHub](#).

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a set of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant

amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Learning PHP, MySQL & JavaScript 6th Edition* by Robin Nixon (O’Reilly). Copyright 2021 Robin Nixon, 9781492093824.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O’Reilly Online Learning



For more than 40 years, *O’Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
(800) 998-9938 (in the United States or Canada)
(707) 829-0515 (international or local)
(707) 829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/learning-php-mysql-js-6e>.

For questions about this book, email bookquestions@oreilly.com.

For news and more information about our books and courses, see our website at <http://www.oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments

I would like to thank Senior Content Acquisitions Editor Amanda Quinn, Content Development Editor Melissa Potter, and everyone who worked so hard on this book, including Michal Špaček and David Mackey for their comprehensive technical reviews, Caitlin Ghegan for overseeing production, Kim Cofer for copy editing, Kim Sandoval for proofreading, Judith McConville for creating the index, Karen Montgomery for the original sugar glider front cover design, Randy Comer for the latest book cover, my original editor, Andy Oram, for overseeing the first five editions, and everyone else too numerous to name who submitted errata and offered suggestions for this new edition.

Introduction to Dynamic Web Content

The World Wide Web is a constantly evolving network that has already traveled far beyond its conception in the early 1990s, when it was created to solve a specific problem. State-of-the-art experiments at CERN (the European Laboratory for Particle Physics, now best known as the operator of the Large Hadron Collider) were producing incredible amounts of data—so much that the data was proving unwieldy to distribute to the participating scientists, who were spread out across the world.

At this time, the internet was already in place, connecting several hundred thousand computers, so Tim Berners-Lee (a CERN fellow) devised a method of navigating between them using a hyperlinking framework, which came to be known as Hypertext Transfer Protocol, or HTTP. He also created a markup language called Hypertext Markup Language, or HTML. To bring these together, he wrote the first web browser and web server.

Today we take these tools for granted, but back then, the concept was revolutionary. The most connectivity so far experienced by at-home modem users was dialing up and connecting to a bulletin board where you could communicate and swap data only with other users of that service. Consequently, you needed to be a member of many bulletin board systems in order to effectively communicate electronically with your colleagues and friends.

But Berners-Lee changed all that in one fell swoop, and by the mid-1990s, there were three major graphical web browsers competing for the attention of five million users. It soon became obvious, though, that something was missing. Yes, pages of text and graphics with hyperlinks to take you to other pages was a brilliant concept, but the results didn't reflect the instantaneous potential of computers and the internet to meet the particular needs of each user with dynamically changing content. Using the web was a very dry and plain experience, even if we did now have scrolling text and animated GIFs!

Shopping carts, search engines, and social networks have clearly altered how we use the web. In this chapter, we'll take a brief look at the various components that make up the web, and the software that helps make using it a rich and dynamic experience.



It is necessary to start using some acronyms more or less right away. I have tried to clearly explain them before proceeding, but don't worry too much about what they stand for or what these names mean, because the details will become clear as you read on.

HTTP and HTML: Berners-Lee's Basics

HTTP is a communication standard governing the requests and responses that are sent between the browser running on the end user's computer and the web server. The server's job is to accept a request from the client and attempt to reply to it in a meaningful way, usually by serving up a requested web page—that's why the term *server* is used. The natural counterpart to a server is a *client*, so that term is applied both to the web browser and the computer on which it's running.

Between the client and the server there can be several other devices, such as routers, proxies, gateways, and so on. They serve different roles in ensuring that the requests and responses are correctly transferred between the client and server. Typically, they use the internet to send this information. Some of these in-between devices can also help speed up the internet by storing pages or information locally in what is called a *cache* and then serving this content up to clients directly from the cache rather than fetching it all the way from the source server.

A web server can usually handle multiple simultaneous connections, and when not communicating with a client, it spends its time listening for an incoming connection. When one arrives, the server sends back a response to confirm its receipt.

The Request/Response Procedure

At its most basic level, the request/response process consists of a web browser or other client asking the web server to send it a web page and the server sending back the page. The browser then takes care of displaying or rendering the page (see [Figure 1-1](#)).

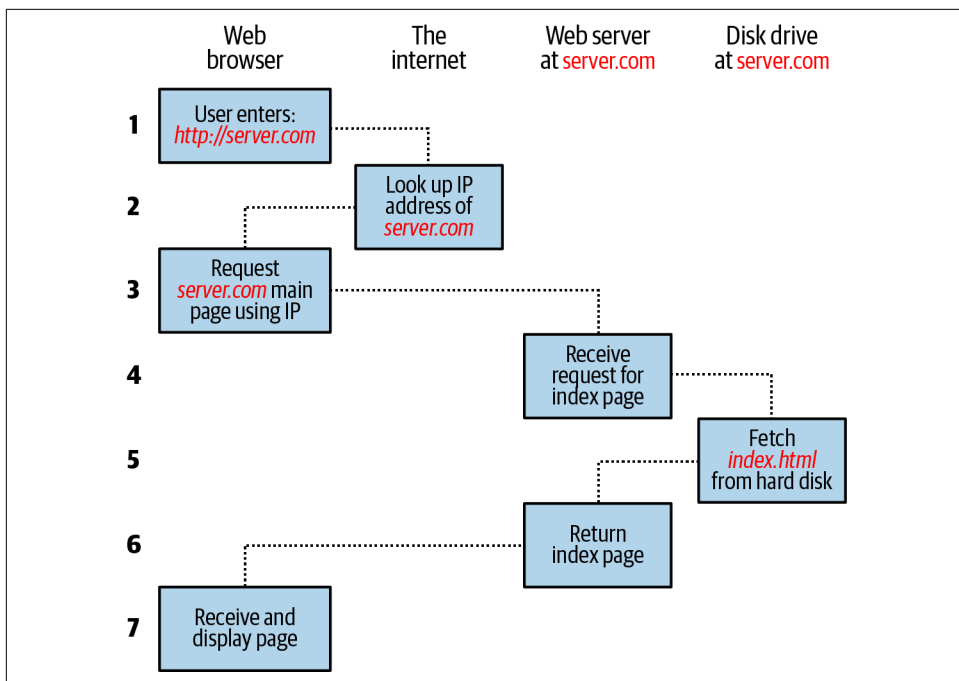


Figure 1-1. The basic client/server request/response sequence

The steps in the request and response sequence are as follows:

1. You enter *http://server.com* into your browser's address bar.
2. Your browser looks up the Internet Protocol (IP) address for *server.com*.
3. Your browser issues a request for the home page at *server.com*.
4. The request crosses the internet and arrives at the *server.com* web server.
5. The web server, having received the request, looks for the web page on its disk.
6. The web server retrieves the page and returns it to the browser.
7. Your browser displays the web page.

For an average web page, this process also takes place once for each object within the page: a graphic, an embedded video or Flash file, and even a CSS template.

In step 2, notice that the browser looks up the IP address of *server.com*. Every machine attached to the internet has an IP address—your computer included—but we generally access web servers by name, such as *google.com*. The browser consults an additional internet service called the Domain Name System (DNS) to find the server's associated IP address and then uses it to communicate with the computer.

For dynamic web pages, the procedure is a little more involved, because it may bring both PHP and MySQL into the mix. For instance, you may click a picture of a raincoat. Then PHP will put together a request using the standard database language, SQL—many of whose commands you will learn in this book—and send the request to the MySQL server. The MySQL server will return information about the raincoat you selected, and the PHP code will wrap it all up in some HTML, which the server will send to your browser (see [Figure 1-2](#)).

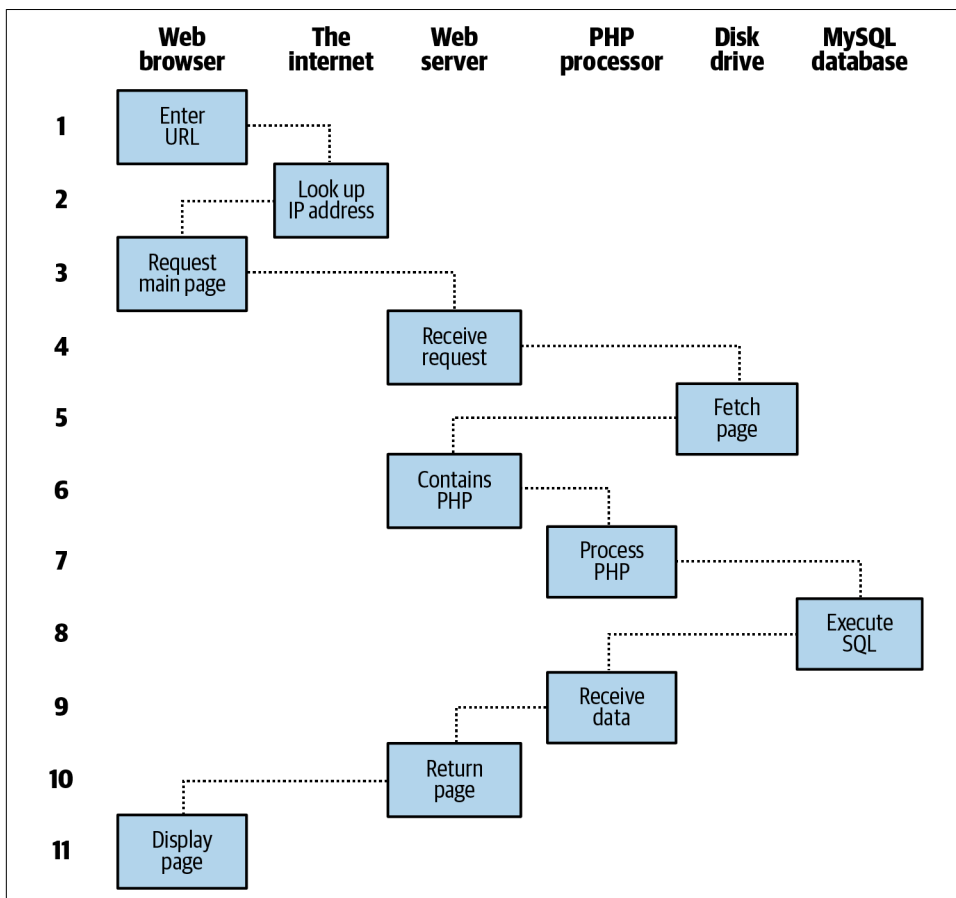


Figure 1-2. A dynamic client/server request/response sequence

The steps are as follows:

1. You enter `http://server.com` into your browser's address bar.
2. Your browser looks up the IP address for `server.com`.
3. Your browser issues a request to that address for the web server's home page.

4. The request crosses the internet and arrives at the *server.com* web server.
5. The web server, having received the request, fetches the home page from its hard disk.
6. With the home page now in memory, the web server notices that it is a file incorporating PHP scripting and passes the page to the PHP interpreter.
7. The PHP interpreter executes the PHP code.
8. Some of the PHP contains SQL statements, which the PHP interpreter now passes to the MySQL database engine.
9. The MySQL database returns the results of the statements to the PHP interpreter.
10. The PHP interpreter returns the results of the executed PHP code, along with the results from the MySQL database, to the web server.
11. The web server returns the page to the requesting client, which displays it.

Although it's helpful to be aware of this process so that you know how the three elements work together, in practice you don't really need to concern yourself with these details, because they all happen automatically.

The HTML pages returned to the browser in each example may well contain JavaScript, which will be interpreted locally by the client, and which could initiate another request—the same way embedded objects such as images would.

The Benefits of PHP, MySQL, JavaScript, CSS, and HTML5

At the start of this chapter, I introduced the world of Web 1.0, but it wasn't long before the rush was on to create Web 1.1, with the development of such browser enhancements as Java, JavaScript, JScript (Microsoft's slight variant of JavaScript), and ActiveX. On the server side, progress was being made on the Common Gateway Interface (CGI) using scripting languages such as Perl (an alternative to the PHP language) and *server-side scripting*—inserting the contents of one file (or the output of running a local program) into another one dynamically.

Once the dust had settled, three main technologies stood head and shoulders above the others. Although Perl was still a popular scripting language with a strong following, PHP's simplicity and built-in links to the MySQL database program had earned it more than double the number of users. And JavaScript, which had become an essential part of the equation for dynamically manipulating Cascading Style Sheets (CSS) and HTML, now took on the even more muscular task of handling the client side of asynchronous communication (exchanging data between a client and server after a web page has loaded). Using asynchronous communication, web pages perform data handling and send requests to web servers in the background—without the web user being aware that this is going on.

No doubt the symbiotic nature of PHP and MySQL helped propel them both forward, but what attracted developers to them in the first place? The simple answer has to be the ease with which you can use them to quickly create dynamic elements on websites. MySQL is a fast and powerful yet easy-to-use database system that offers just about anything a website would need in order to find and serve up data to browsers. When PHP allies with MySQL to store and retrieve this data, you have the fundamental parts required for the beginnings of Web 2.0.

And when you bring JavaScript and CSS into the mix too, you have a recipe for building highly dynamic and interactive websites—especially as there is now a wide range of sophisticated frameworks of JavaScript functions you can call on to really speed up web development. These include the well-known jQuery, which until very recently was one of the most common ways programmers accessed asynchronous communication features and the more recent React JavaScript library, which has been growing quickly in popularity. It is now one of the most widely downloaded and implemented frameworks, so much so that since 2020 the Indeed job site lists more than twice as many positions for React developers than for jQuery.

MariaDB: The MySQL Clone

After Oracle purchased Sun Microsystems (the owners of MySQL), the community became wary that MySQL might not remain fully open source, so MariaDB was forked from it to keep it free under the GNU GPL. Development of MariaDB is led by some of the original developers of MySQL, and it retains exceedingly close compatibility with MySQL. Therefore, you may well encounter MariaDB on some servers in place of MySQL—but not to worry, everything in this book works equally well on both MySQL and MariaDB. For all intents and purposes, you can swap one with the other and notice no difference.

Anyway, as it turns out, many of the initial fears appear to have been allayed as MySQL remains open source, with Oracle simply charging for support and for editions that provide additional features such as geo-replication and automatic scaling. However, unlike MariaDB, MySQL is no longer community driven, so knowing that MariaDB will always be there if ever needed will keep many developers sleeping at night and probably ensures that MySQL itself will remain open source.

Using PHP

With PHP, it's a simple matter to embed dynamic activity in web pages. When you give pages the *.php* extension, they have instant access to the scripting language. From a developer's point of view, all you have to do is write code such as the following:

```
<?php
    echo " Today is " . date("l") . ". ";
?>
```

Here's the latest news.

The opening `<?php` tells the web server to allow the PHP program to interpret all the following code up to the `?>` tag. Outside of this construct, everything is sent to the client as direct HTML. So, the text `Here's the latest news.` is simply output to the browser; within the PHP tags, the built-in `date` function displays the current day of the week according to the server's system time.

The final output of the two parts looks like this:

Today is Wednesday. Here's the latest news.

PHP is a flexible language, and some people prefer to place the PHP construct directly next to PHP code, like this:

`Today is <?php echo date("l"); ?>. Here's the latest news.`

There are even more ways of formatting and outputting information, which I'll explain in the chapters on PHP. The point is that with PHP, web developers have a scripting language that, although not as fast as compiling your code in C or a similar language, is incredibly speedy and also integrates seamlessly with HTML markup.



If you intend to enter the PHP examples in this book into a program editor to work along with me, you must remember to add `<?php` in front and `?>` after them to ensure that the PHP interpreter processes them. To facilitate this, you may wish to prepare a file called *example.php* with those tags in place.

Using PHP, you have unlimited control over your web server. Whether you need to modify HTML on the fly, process a credit card, add user details to a database, or fetch information from a third-party website, you can do it all from within the same PHP files in which the HTML itself resides.

Using MySQL

Of course, there's not a lot of point to being able to change HTML output dynamically unless you also have a means to track the information users provide to your website as they use it. In the early days of the web, many sites used “flat” text files to store data such as usernames and passwords. But this approach could cause problems if the file wasn't correctly locked against corruption from multiple simultaneous accesses. Also, a flat file can get only so big before it becomes unwieldy to manage—not to mention the difficulty of trying to merge files and perform complex searches in any kind of reasonable time.

That's where relational databases with structured querying become essential. And MySQL, being free to use and installed on vast numbers of internet web servers, rises

superbly to the occasion. It is a robust and exceptionally fast database management system that uses English-like commands.

The highest level of MySQL structure is a database, within which you can have one or more tables that contain your data. For example, let's suppose you are working on a table called *users*, within which you have created columns for *surname*, *firstname*, and *email*, and you now wish to add another user. One command that you might use to do this is as follows:

```
INSERT INTO users VALUES('Smith', 'John', 'jsmith@mysite.com');
```

You will previously have issued other commands to create the database and table and to set up all the correct fields, but the SQL INSERT command here shows how simple it can be to add new data to a database. SQL is a language designed in the early 1970s that is reminiscent of one of the oldest programming languages, COBOL. It is well suited, however, to database queries, which is why it is still in use after all this time.

It's equally easy to look up data. Let's assume that you have an email address for a user and need to look up that person's name. To do this, you could issue a MySQL query such as the following:

```
SELECT surname,firstname FROM users WHERE email='jsmith@mysite.com';
```

MySQL will then return Smith, John and any other pairs of names that may be associated with that email address in the database.

As you'd expect, there's quite a bit more that you can do with MySQL than just simple INSERT and SELECT commands. For example, you can combine related data sets to bring related pieces of information together, ask for results in a variety of orders, make partial matches when you know only part of the string that you are searching for, return only the *n*th result, and a lot more.

Using PHP, you can make all these calls directly to MySQL without having to directly access the MySQL command-line interface yourself. This means you can save the results in arrays for processing and perform multiple lookups, each dependent on the results returned from earlier ones, to drill down to the item of data you need.

For even more power, as you'll see later, there are additional functions built right into MySQL that you can call up to efficiently run common operations within MySQL, rather than creating them out of multiple PHP calls to MySQL.

Using JavaScript

JavaScript was created to enable scripting access to all the elements of an HTML document. In other words, it provides a means for dynamic user interaction such as checking email address validity in input forms and displaying prompts such as “Did you really mean that?” (although it cannot be relied upon for security, which should always be performed on the web server).

Combined with CSS (see the following section), JavaScript is the power behind dynamic web pages that change in front of your eyes rather than when a new page is returned by the server.

However, JavaScript used to be tricky to use, due to some major differences in the ways different browser designers have chosen to implement it. This mainly came about when some manufacturers tried to put additional functionality into their browsers at the expense of compatibility with their rivals.

Thankfully, the developers have mostly come to their senses and have realized the need for full compatibility with one another, so it is less necessary these days to have to optimize your code for different browsers. However, there remain millions of users using legacy browsers, and this will likely be the case for a good many years to come. Luckily, there are solutions for the incompatibility problems, and later in this book we'll look at libraries and techniques that enable you to safely ignore these differences.

For now, let's take a look at how to use basic JavaScript, accepted by all browsers:

```
<script type="text/javascript">
  document.write("Today is " + Date() );
</script>
```

This code snippet tells the web browser to interpret everything within the `<script>` tags as JavaScript, which the browser does by writing the text `Today is` to the current document, along with the date, using the JavaScript function `Date`. The result will look something like this:

Today is Wed Jan 01 2025 01:23:45



Unless you need to specify an exact version of JavaScript, you can normally omit the `type="text/javascript"` and just use `<script>` to start the interpretation of the JavaScript.

As previously mentioned, JavaScript was originally developed to offer dynamic control over the various elements within an HTML document, and that is still its main use. But more and more, JavaScript is being used for *Ajax*, the process of accessing the web server in the background.

Asynchronous communication is what allows web pages to begin to resemble stand-alone programs, because they don't have to be reloaded in their entirety to display new content. Instead, an asynchronous call can pull in and update a single element on a web page, such as changing your photograph on a social networking site or replacing a button that you click with the answer to a question. This subject is fully covered in [Chapter 18](#).

Then, in [#introduction_to_jquery](#), we take a good look at the jQuery framework, which you can use to save reinventing the wheel when you need fast, cross-browser code to manipulate your web pages. Of course, there are other frameworks available too, so we also take a look at React, one of the most popular choices of today, in [Chapter 24](#). Both are extremely reliable and are major tools in the utility kits of many seasoned web developers.

Using CSS

CSS is the crucial companion to HTML, ensuring that the HTML text and embedded images are laid out consistently and in a manner appropriate for the user's screen. With the emergence of the CSS3 standard in recent years, CSS now offers a level of dynamic interactivity previously supported only by JavaScript. For example, not only can you style any HTML element to change its dimensions, colors, borders, spacing, and so on, but now you can also add animated transitions and transformations to your web pages, using only a few lines of CSS.

Using CSS can be as simple as inserting a few rules between `<style>` and `</style>` tags in the head of a web page, like this:

```
<style>
  p {
    text-align:justify;
    font-family:Helvetica;
  }
</style>
```

These rules change the default text alignment of the `<p>` tag so that paragraphs contained in it are fully justified and use the Helvetica font.

As you'll learn in [Chapter 19](#), there are many different ways you can lay out CSS rules, and you can also include them directly within tags or save a set of rules to an external file to be loaded in separately. This flexibility not only lets you style your HTML precisely but can also (for example) provide built-in hover functionality to animate objects as the mouse passes over them. You will also learn how to access all of an element's CSS properties from JavaScript as well as HTML.

And Then There's HTML5

As useful as all these additions to the web standards became, they were not enough for ever-more ambitious developers. For example, there was still no simple way to manipulate graphics in a web browser without resorting to plug-ins such as Flash. And the same went for inserting audio and video into web pages. Plus, several annoying inconsistencies had crept into HTML during its evolution.

So, to clear all this up and take the internet beyond Web 2.0 and into its next iteration, a new standard for HTML was created to address all these shortcomings: *HTML5*. Its development began as long ago as 2004, when the first draft was drawn up by the Mozilla Foundation and Opera Software (developers of two popular web browsers), but it wasn't until the start of 2013 that the final draft was submitted to the World Wide Web Consortium (W3C), the international governing body for web standards.

It has taken a few years for HTML5 to develop, but now we are at a very solid and stable version 5.1 (since 2016). It's a never-ending cycle of development, though, and more functionality is sure to be built into it over time, with version 5.2 (planned to make the plug-in system obsolete) released as a W3C recommendation in 2017, HTML 5.3 (with proposed features such as auto-capitalization) still in planning as of 2020, and so on. Some of the best features in HTML5 for handling and displaying media include the `<audio>`, `<video>`, and `<canvas>` elements, which add sound, video, and advanced graphics. Everything you need to know about these and all other aspects of HTML5 is covered in detail starting in [Chapter 25](#).



One of the little things I like about the HTML5 specification is that XHTML syntax is no longer required for self-closing elements. In the past, you could display a line break using the `<br>` element. Then, to ensure future compatibility with XHTML (the planned replacement for HTML that never happened), this was changed to `<br />`, in which a closing `/` character was added (since all elements were expected to include a closing tag featuring this character). But now things have gone full circle, and you can use either version of these types of elements. So, for the sake of brevity and fewer keystrokes, in this book I have reverted to the former style of `<br>`, `<hr>`, and so on.

The Apache Web Server

In addition to PHP, MySQL, JavaScript, CSS, and HTML5, there's a sixth hero in the dynamic web: the web server. In the case of this book, that means the Apache web server. We've discussed a little of what a web server does during the HTTP server/client exchange, but it does much more behind the scenes.

For example, Apache doesn't serve up just HTML files—it handles a wide range of files, from images and Flash files to MP3 audio files, RSS (Really Simple Syndication) feeds, and so on. And these objects don't have to be static files such as GIF images. They can all be generated by programs such as PHP scripts. That's right: PHP can even create images and other files for you, either on the fly or in advance to serve up later.

To do this, you normally have modules either precompiled into Apache or PHP or called up at runtime. One such module is the GD (Graphics Draw) library, which PHP uses to create and handle graphics.

Apache also supports a huge range of modules of its own. In addition to the PHP module, the most important for your purposes as a web programmer are the modules that handle security. Other examples are the Rewrite module, which enables the web server to handle a range of URL types and rewrite them to its own internal requirements, and the Proxy module, which you can use to serve up often-requested pages from a cache to ease the load on the server.

Later in the book, you'll see how to use some of these modules to enhance the features provided by the three core technologies.

Handling Mobile Devices

We are now firmly in a world of interconnected mobile computing devices, and the concept of developing websites solely for desktop computers has become rather dated. Instead, developers now aim to develop responsive websites and web apps that tailor themselves to the environment in which they find themselves running.

So, new in this edition, I show how you can easily create these types of products using just the technologies detailed in this book, along with the powerful jQuery Mobile library of responsive JavaScript functions. With it, you'll be able to focus on the content and usability of your websites and web apps, knowing that how they display will be automatically optimized for a range of different computing devices—one less thing for you to worry about.

To demonstrate how to make full use of its power, the final chapter of this book creates a simple social networking example website, using jQuery to make it fully responsive and ensure it displays well on anything from a small mobile phone screen to a tablet or a desktop computer. We could equally have used React (or other JavaScript libraries or frameworks), but perhaps that's an exercise you'll like to set yourself once you've completed this book.

About Open Source

The technologies in this book are open source: anyone is allowed to read and change the code. Whether or not this status is the reason these technologies are so popular has often been debated, but PHP, MySQL, and Apache *are* the three most commonly used tools in their categories. What can be said definitively, though, is that their being open source means that they have been developed in the community by teams of programmers writing the features they themselves want and need, with the original code

available for all to see and change. Bugs can be found quickly, and security breaches can be prevented before they happen.

There's another benefit: all of these programs are free to use. There's no worrying about having to purchase additional licenses if you have to scale up your website and add more servers, and you don't need to check the budget before deciding whether to upgrade to the latest versions of these products.

Bringing It All Together

The real beauty of PHP, MySQL, JavaScript (sometimes aided by React or other frameworks), CSS, and HTML5 is the wonderful way in which they all work together to produce dynamic web content: PHP handles all the main work on the web server, MySQL manages all the data, and the combination of CSS and JavaScript looks after web page presentation. JavaScript can also talk with your PHP code on the web server whenever it needs to update something (either on the server or on the web page). And with the powerful features in HTML5, such as the canvas, audio and video, and geolocation, you can make your web pages highly dynamic, interactive, and multimedia-packed.

Without using program code, let's summarize the contents of this chapter by looking at the process of combining some of these technologies into an everyday asynchronous communication feature that many websites use: checking whether a desired username already exists on the site when a user is signing up for a new account. A good example of this can be seen with Gmail (see [Figure 1-3](#)).

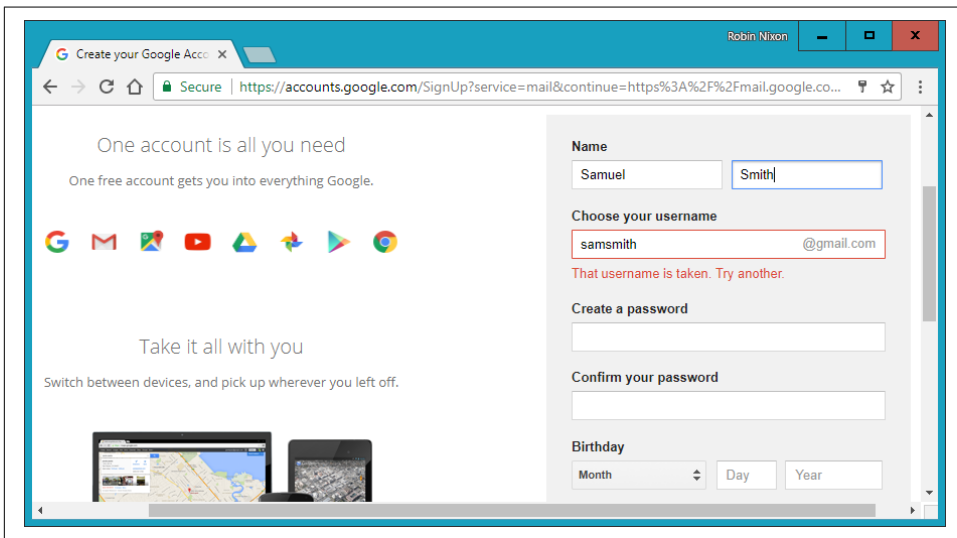


Figure 1-3. Gmail uses asynchronous communication to check the availability of usernames

The steps involved in this asynchronous process will be similar to the following:

1. The server outputs the HTML to create the web form, which asks for the necessary details, such as username, first name, last name, and email address.
2. At the same time, the server attaches some JavaScript to the HTML to monitor the username input box and check for two things: whether some text has been typed into it, and whether the input has been deselected because the user has clicked another input box.
3. Once the text has been entered and the field deselected, in the background the JavaScript code passes the username that was entered back to a PHP script on the web server and awaits a response.
4. The web server looks up the username and replies back to the JavaScript regarding whether that name has already been taken.
5. The JavaScript then places an indication next to the username input box to show whether the name is available to the user—perhaps a green checkmark or a red cross graphic, along with some text.
6. If the username is not available and the user still submits the form, the JavaScript interrupts the submission and reemphasizes (perhaps with a larger graphic and/or an alert box) that the user needs to choose another username.
7. Optionally, an improved version of this process could even look at the username requested by the user and suggest an alternative that is currently available.

All of this takes place quietly in the background and makes for a comfortable and seamless user experience. Without asynchronous communication, the entire form would have to be submitted to the server, which would then send back HTML, highlighting any mistakes. It would be a workable solution but nowhere near as tidy or pleasurable as on-the-fly form field processing.

Asynchronous communication can be used for a lot more than simple input verification and processing, though; we'll explore many additional things that you can do with it later in this book.

In this chapter, you have read a good introduction to the core technologies of PHP, MySQL, JavaScript, CSS, and HTML5 (as well as Apache) and have learned how they work together. In [Chapter 2](#), we'll look at how you can install your own web development server on which to practice everything that you will be learning.

Questions

1. What four components (at the minimum) are needed to create a fully dynamic web page?
2. What does *HTML* stand for?
3. Why does the name *MySQL* contain the letters *SQL*?
4. *PHP* and *JavaScript* are both programming languages that generate dynamic results for web pages. What is their main difference, and why would you use both of them?
5. What does *CSS* stand for?
6. List three major new elements introduced in HTML5.
7. If you encounter a bug (which is rare) in one of the open source tools, how do you think you could get it fixed?
8. Why is a framework such as jQuery or React so important for developing modern websites and web apps?

See “Chapter 1 Answers” on page 747 in the [Appendix](#) for the answers to these questions.

Setting Up a Development Server

If you wish to develop internet applications but don't have your own development server, you will have to upload every modification you make to a server somewhere else on the web before you can test it.

Even on a fast broadband connection, this can still represent a significant slowdown in development time. On a local computer, however, testing can be as easy as saving an update (usually just a matter of clicking once on an icon) and then hitting the Refresh button in your browser.

Another advantage of a development server is that you don't have to worry about embarrassing errors or security problems while you're writing and testing, whereas you need to be aware of what people may see or do with your application when it's on a public website. It's best to iron everything out while you're still on a home or small office system, presumably protected by firewalls and other safeguards.

Once you have your own development server, you'll wonder how you ever managed without one, and it's easy to set one up. Just follow the steps in the following sections, using the appropriate instructions for a PC, a Mac, or a Linux system.

In this chapter, we cover just the server side of the web experience, as described in [Chapter 1](#). But to test the results of your work—particularly when we start using JavaScript, CSS, and HTML5 later in this book—you should ideally have an instance of every major web browser running on some system convenient to you. Whenever possible, the list of browsers should include at least Microsoft Edge, Mozilla Firefox, Opera, Safari, and Google Chrome. You may need all these once you have a product ready to release, just to ensure everything runs as expected on all browsers and platforms. If you plan to ensure that your sites look good on mobile devices too, you should try to arrange access to a wide range of iOS and Android devices.

What Is a WAMP, MAMP, or LAMP?

WAMP, MAMP, and LAMP are abbreviations for “Windows, Apache, MySQL, and PHP,” “Mac, Apache, MySQL, and PHP,” and “Linux, Apache, MySQL, and PHP.” These abbreviations each describe a fully functioning setup used for developing dynamic internet web pages.

WAMPs, MAMPs, and LAMPs come in the form of packages that bind the bundled programs together so that you don’t have to install and set them up separately. This means you can simply download and install a single program and follow a few easy prompts to get your web development server up and running fast, with minimal hassle.

During installation, several default settings are created for you. The security configurations of such an installation will not be as tight as on a production web server, because it is optimized for local use. For these reasons, you should never install such a setup as a production server.

However, for developing and testing websites and applications, one of these installations should be entirely sufficient.



If you choose not to go the WAMP/MAMP/LAMP route for building your own development system, you should know that downloading and integrating the various parts yourself can be very time-consuming and may require a lot of research in order to configure everything fully. But if you already have all the components installed and integrated with one another, they should work with the examples in this book.

Installing AMPPS on Windows

There are several available WAMP servers, each offering slightly different configurations. Of the various open source and free options, one of the best is AMPPS. You can download it by clicking the button on the website’s [home page](#), shown in [Figure 2-1](#). (There are also Mac and Linux versions available; see “[Installing AMPPS on macOS](#)” on page 27 and “[Installing a LAMP on Linux](#)” on page 29.)



Just recently Chrome has been updated to disallow downloads from mixed sources (such as an `http://` file from an `https://` web page). It is likely that other browsers will follow this security initiative too. Currently the AMPPS website uses mixed sources, and you may encounter this issue. The answer is to not select *DISCARD* when Chrome (or another browser) prompts that “AMPPS *can’t be downloaded securely*” and to use the up selection arrow to select the option *Keep*, and your download will resume. Additionally, if clicking the download link appears to do nothing, you will have to right-click it and select *Save As* to initiate the download.

I recommend that you always download the latest stable release (as I write this, it’s 3.9, which is about 114 MB in size). The various Windows, macOS, and Linux installers are listed on the download page.

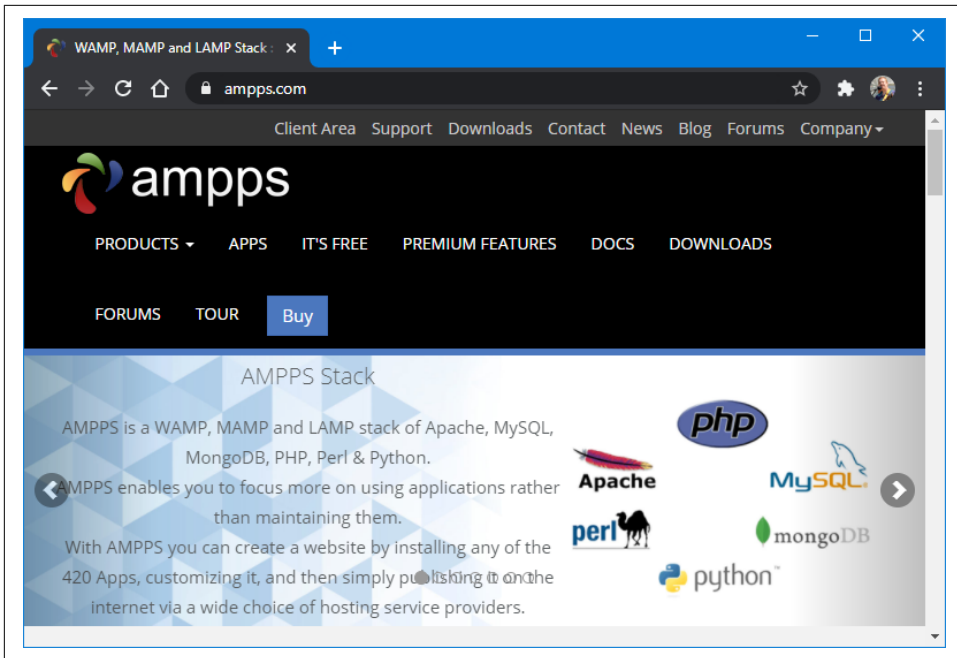


Figure 2-1. The AMPPS website



During the lifetime of this edition, some of the screens and options shown in the following walk-through may change. If so, just use your common sense to proceed in as similar a manner as possible to the sequence of actions described.

Once you've downloaded the installer, run it to bring up the window shown in **Figure 2-2**. Before arriving at that window, though, if you use an antivirus program or have User Account Control activated on Windows, you may first be shown one or more advisory notices, and will have to click Yes and/or OK to continue with the installation.

Click Next, after which you must accept the agreement. Click Next once again, and then once more to move past the information screen. You will now need to confirm the installation location. This will probably be suggested as something like the following, depending on the letter of your main hard drive, but you can change this if you wish:

C:\Program Files (x86)\Ampps



Figure 2-2. The opening window of the installer

Next you must accept the agreements in the following screen and click Next, then after reading the information summary click Next once more and you will be asked which folder you wish to install AMPPS into.

Once you have decided where to install AMPPS, click Next, decide where shortcuts should be saved (the default shown is usually just fine), and click Next again to choose which icons you wish to install, as shown in **Figure 2-3**. On the screen that follows, click the Install button to start the process.

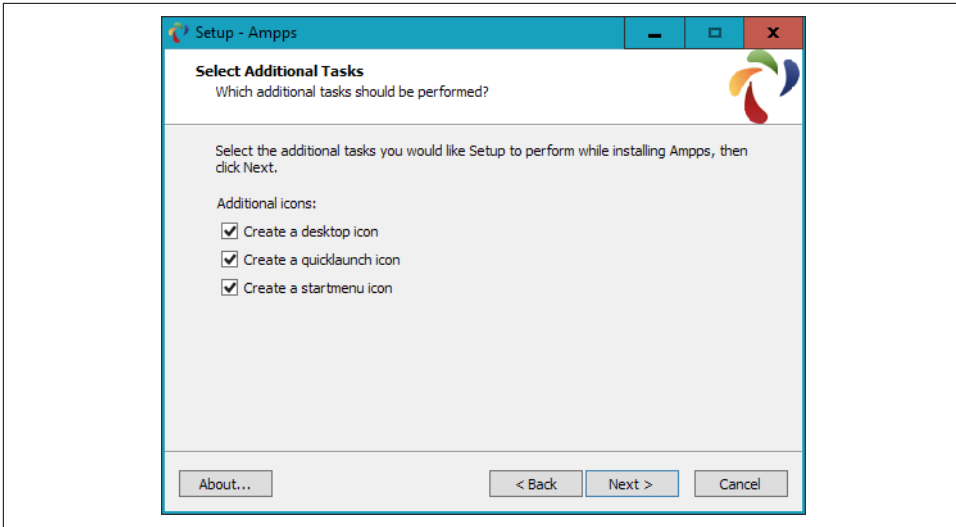


Figure 2-3. Choose which icons to install

Installation will take a few minutes, after which you should see the completion screen in [Figure 2-4](#), and you can click Finish.



Figure 2-4. AMPPS is now installed

The final thing you must do is install Microsoft Visual C++ Redistributable, if you haven't already. A window will pop up to prompt you, as shown in [Figure 2-5](#). Click Yes to start the installation or No if you are certain you already have it. Or, you can always proceed anyway, and you will be told whether you don't need to reinstall it.

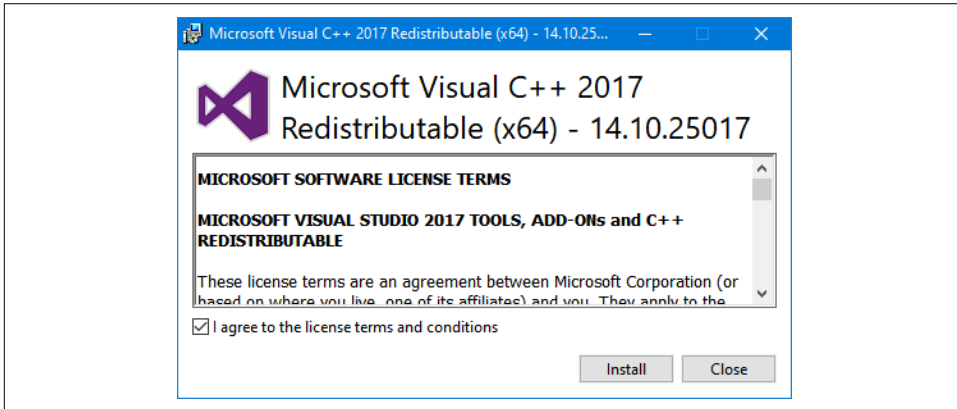


Figure 2-5. Install Visual C++ Redistributable if you don't already have it

If you choose to go ahead and install, you will have to agree to the terms and conditions in the pop-up window that appears and then click Install. Installation of this should be fairly fast. Click Close to finish.

Once AMPPS is installed, the control window shown in [Figure 2-6](#) should appear at the bottom right of your desktop. You can also call up this window using the AMPPS application shortcut in the Start menu or on the desktop, if you allowed these icons to be created.

Before proceeding, if you have any further questions, I recommend you acquaint yourself with the [AMPPS documentation](#), otherwise you are set to go—there's always a Support link at the bottom of the control window that will take you to the AMPPS website, where you can open up a trouble ticket should you need to.

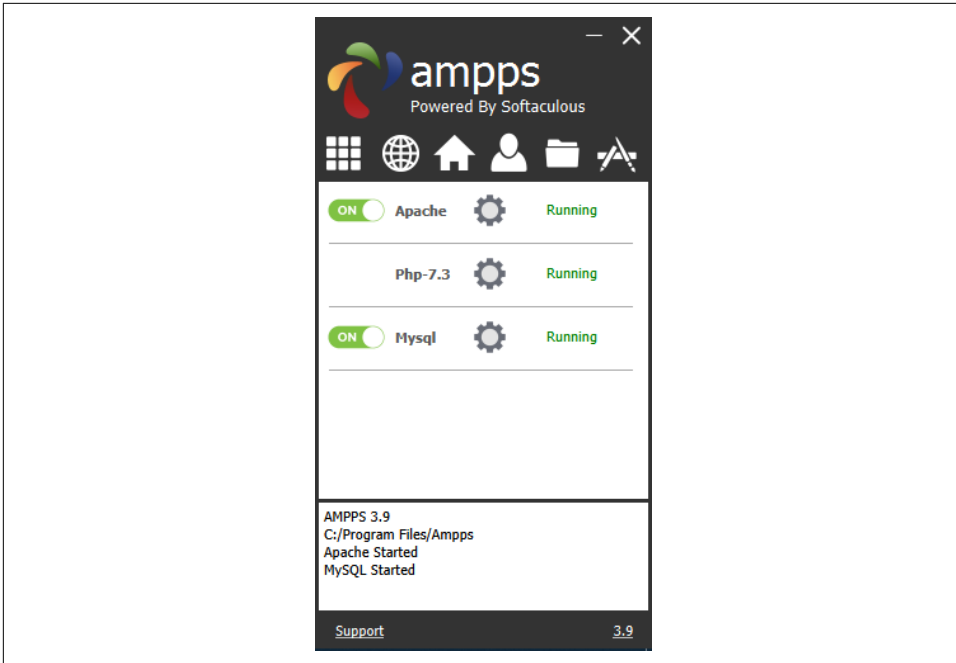


Figure 2-6. The AMPPS control window



You may notice that the default version of PHP in AMPPS is 7.3. If you wish to try version 5.6 for any reason, click the Options button (nine white boxes in a square) within the AMPPS control window and then select Change PHP Version, whereupon a new menu will appear from which you can choose a version between 5.6 and 7.3.

Testing the Installation

The first thing to do at this point is verify that everything is working correctly. To do this, enter either of the following two URLs into the address bar of your browser:

```
localhost
127.0.0.1
```

This will call up an introductory screen, where you will have the opportunity to secure AMPPS by giving it a password (see [Figure 2-7](#)). I recommend you don't check the box and just click the Submit button to proceed without setting a password.

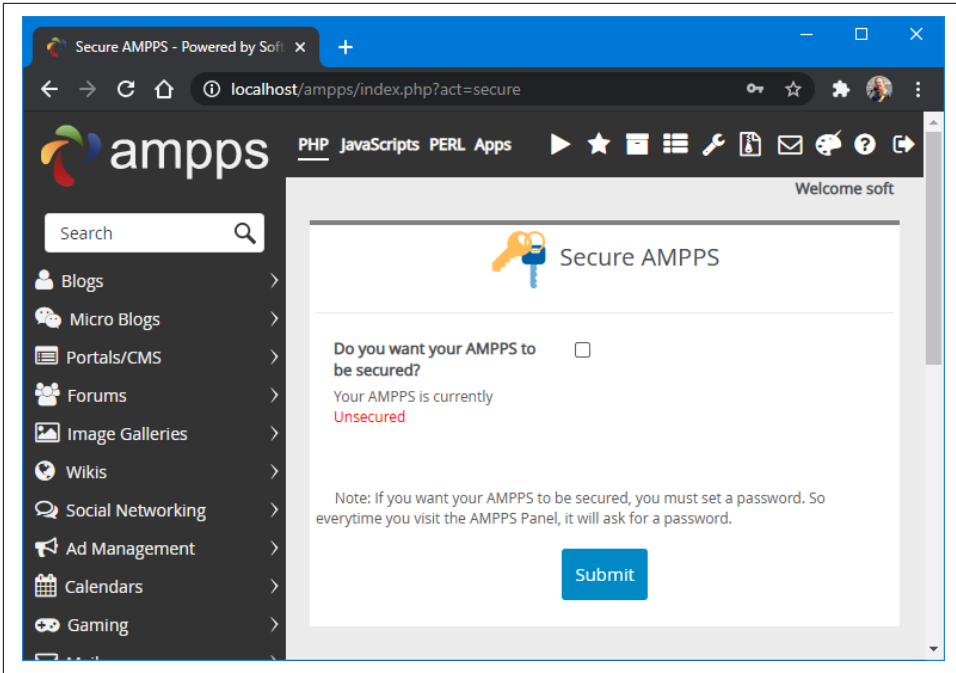


Figure 2-7. The initial security setup screen

Once this has been done you will be taken to the main control page at *localhost/ampps/* (from now on I will assume you are accessing AMPPS through *localhost* rather than *127.0.0.1*). From here you can configure and control all aspects of the AMPPS stack, so make a note of this for future reference, or perhaps set a bookmark in your browser.

Next, type the following to view the document root (described in the following section) of your new Apache web server:

```
localhost
```

This time, rather than seeing the initial screen about setting up security, you should see something similar to [Figure 2-8](#).

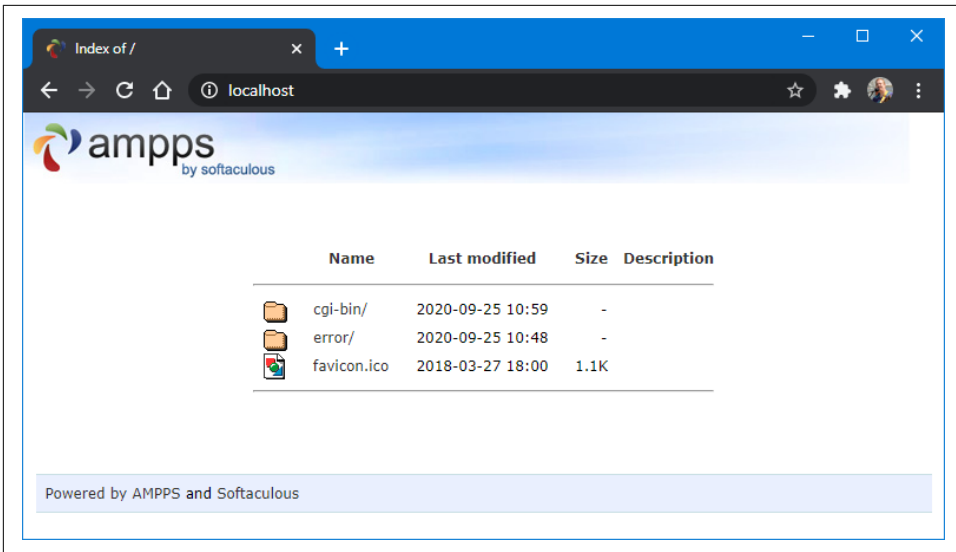


Figure 2-8. Viewing the document root

Accessing the Document Root (Windows)

The *document root* is the directory that contains the main web documents for a domain. This directory is the one that the server uses when a basic URL without a path is typed into a browser, such as <http://yahoo.com> or, for your local server, <http://localhost>.

By default AMPPS will use the following location as the document root:

```
C:\Program Files\Ampps\www
```

To ensure that you have everything correctly configured, you should now create the obligatory “Hello World” file. So, create a small HTML file along the following lines using Windows Notepad (or *Notepad++* on Windows, or *Atom* on a Mac, or your choice of many others available but not a rich word processor such as Microsoft Word):

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>A quick test</title>
  </head>
  <body>
    Hello World!
  </body>
</html>
```

Once you have typed this, save the file into the document root directory, using the filename *test.html*. If you are using Notepad, make sure that the value in the “Save as type” box is changed from Text Documents (*.txt) to All Files (*.*)).

You can now call this page up in your browser by entering the following URL in its address bar (see [Figure 2-9](#)):

`localhost/test.html`

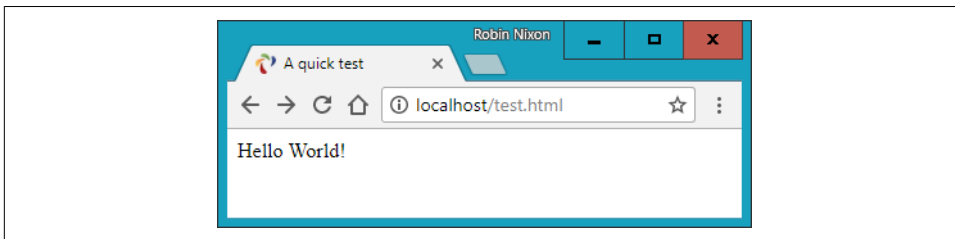


Figure 2-9. Your first web page



Remember that serving a web page from the document root (or a subfolder) is different from loading one into a web browser from your computer’s filesystem. The former will ensure access to PHP, MySQL, and all the features of a web server, while the latter will simply load the file into the browser, which will do its best to display it but will be unable to process any PHP or other server instructions. So, you should generally run examples using the *localhost* preface from your browser’s address bar, unless you are certain that the file doesn’t rely on web server functionality.

Alternative WAMPs

When software is updated, it sometimes works differently from how you expect, and bugs can even be introduced. So, if you encounter difficulties that you cannot resolve in AMPPS, you may prefer to choose one of the other solutions available on the web.

You will still be able to make use of all the examples in this book, but you’ll have to follow the instructions supplied with each WAMP, which may not be as easy to follow as the preceding guide.

Here’s a selection of some of the best, in my opinion:

- EasyPHP
- XAMPP
- WAMPServer



Over the life of this edition of the book, it is very likely that the developers of AMPPS will make improvements to the software, and therefore the installation screens and method of use may evolve over time, as may versions of Apache, PHP, or MySQL. So, please don't assume something is wrong if the screens and operation look different. The AMPPS developers take every care to ensure it is easy to use, so just follow any prompts given, and refer to the documentation on the [website](#).

Installing AMPPS on macOS

AMPPS is also available on macOS, and you can download it from the [website](#), as shown previously in [Figure 2-1](#) (as I write, the current version is 3.0, and its size is around 270 MB).

If your browser doesn't open it automatically once it has downloaded, double-click the *.dmg* file, and then drag the *AMPPS* folder over to your *Applications* folder (see [Figure 2-10](#)).

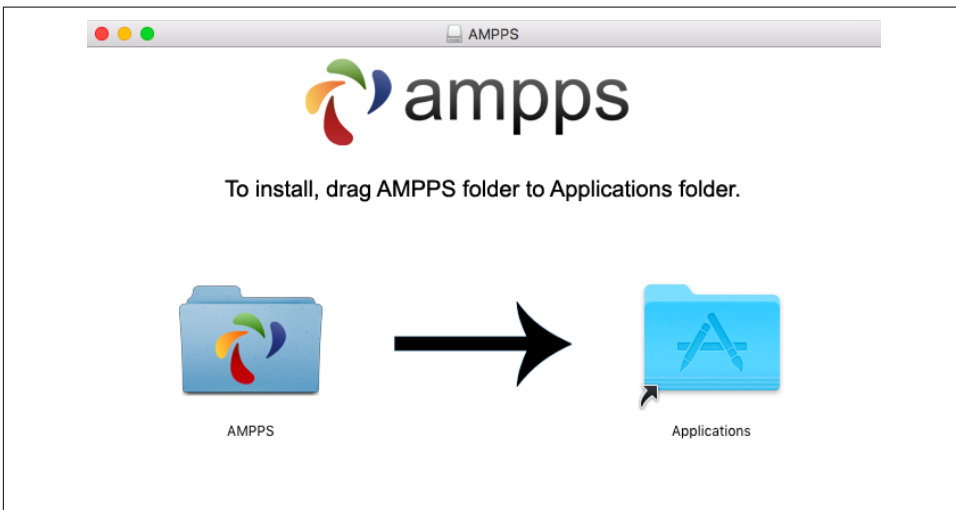


Figure 2-10. Drag the AMPPS folder to Applications

Now open your *Applications* folder in the usual manner, and double-click the AMPPS program. If your security settings prevent the file being opened, hold down the Control key and click the icon once. A new window will pop up asking if you are sure you wish to open it. Click Open to do so. When the app starts, you may have to enter your macOS password to proceed.

Once AMPPS is up and running, a control window similar to the one shown in [Figure 2-6](#) will appear at the bottom left of your desktop.



You may notice that the default version of PHP in AMPPS is 7.3. If you wish to try out version 5.6 for any reason, click the Options button (nine white boxes in a square) within the AMPPS control window, and then select Change PHP Version, whereupon a new menu will appear in which you can choose a version between 5.6 and 7.3.

Accessing the Document Root (macOS)

By default, AMPPS will use the following location as the document root:

```
/Applications/Ampps/www
```

To ensure that you have everything correctly configured, you should now create the obligatory “Hello World” file. So, create a small HTML file along the following lines using the TextEdit program or any other program or text editor but not a rich word processor such as Microsoft Word:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>A quick test</title>
  </head>
  <body>
    Hello World!
  </body>
</html>
```

Once you have typed this, save the file into the document root directory using the filename *test.html*.

You can now call this page up in your browser by entering the following URL in its address bar (to see a similar result to [Figure 2-9](#)):

```
localhost/test.html
```



Remember that serving a web page from the document root (or a subfolder) is different from loading one into a web browser from your computer's filesystem. The former will ensure access to PHP, MySQL, and all the features of a web server, while the latter will simply load the file into the browser, which will do its best to display it but will be unable to process any PHP or other server instructions. So, you should generally run examples using the *localhost* preface from your browser's address bar, unless you are certain that the file doesn't rely on web server functionality.

Installing a LAMP on Linux

This book is aimed mostly at PC and Mac users, but its contents will work equally well on a Linux computer. However, there are dozens of popular flavors of Linux, and each of them may require installing a LAMP in a slightly different way, so I can't cover them all in this book. However, there is a version of AMPPS available for Linux, which is probably the simplest way for you to go.

That said, some Linux versions come preinstalled with a web server and MySQL, and the chances are that you may already be all set to go. To find out, try entering the following into a browser and see whether you get a default document root web page:

`localhost`

If this works, you probably have the Apache server installed and may well have MySQL up and running too; check with your system administrator to be sure.

If you don't yet have a web server installed, however, there's a version of AMPPS available that you can download from the website.

Installation is similar to the sequence shown in the preceding section. If you need further assistance on using the software, please refer to the documentation.

Working Remotely

If you have access to a web server already configured with PHP and MySQL, you can always use that for your web development. But unless you have a high-speed connection, it is not always your best option. Developing locally allows you to test modifications with little or no upload delay.

Accessing MySQL remotely may not be easy either. You should use the secure SSH protocol to log in to your server to manually create databases and set permissions from the command line. Your web hosting company will advise you on how best to do this and provide you with any password it has set for your MySQL access (as well as, of course, for getting into the server in the first place). I recommend you never use the insecure Telnet protocol to remotely log in to any server.

Logging In

I recommend that, at minimum, Windows users should install a program such as **PuTTY** for SSH access (remember that SSH is much more secure than Telnet).

On a Mac, you already have SSH available. Just select the *Applications* folder, followed by *Utilities*, and then launch Terminal. In the Terminal window, log in to a server using SSH as follows:

```
ssh mylogin@server.com
```

where *server.com* is the name of the server you wish to log in to and *mylogin* is the username you will log in under. You will then be prompted for the correct password for that username and, if you enter it correctly, you will be logged in.

Using SFTP or FTPS

To transfer files to and from your web server, you will generally need an FTP, SFTP, or FTPS program. Although FTP is not a secure protocol, software that helps you upload and download files is still often referred to as FTP, but it's FTPS or SFTP that you need to ensure proper security on your web server. If you go searching the web for a good client, you'll find so many that it could take you quite a while to come across one with all the right features for you.



Don't Use FTP

FTP is insecure and should not be used. There are far safer methods than FTP for transferring files, such as using Git or similar technologies. Also, SSH-based SFTP (Secure File Transfer Protocol) and SCP (Secure Copy Protocol) are gaining traction. Good FTP programs, however, will also support SFTP and FTPS (FTP-SSL). Often the means of file transfer you use will be down to the policies of the company you work for, but for personal use an FTP program such as FileZilla (discussed next) will provide most (if not all) of the functionality and security you require.

My preferred FTP/SFTP program is the open source **FileZilla**, for Windows, Linux, and macOS 10.5 or newer (see **Figure 2-11**). Full instructions on how to use FileZilla are available on the [wiki](#).

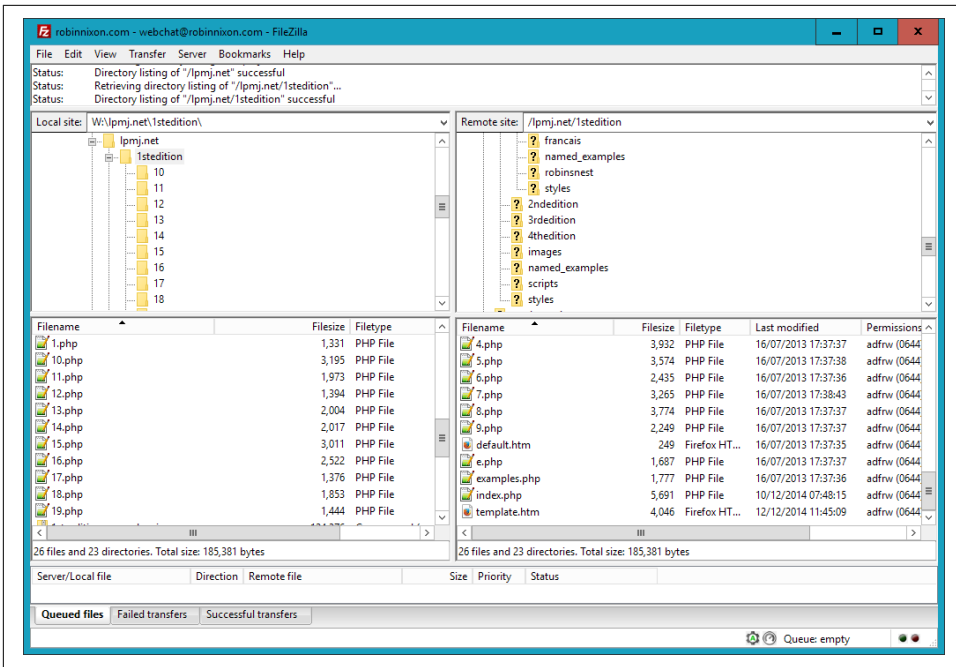


Figure 2-11. FileZilla is a full-featured FTP program

Of course, if you already have an FTPS or SFTP program, all the better—stick with what you know.

Using a Code Editor

Although a plain-text editor works for editing HTML, PHP, and JavaScript, there have been some tremendous improvements in dedicated code editors, which now incorporate very handy features such as colored syntax highlighting. Today's program editors are smart and can show you where you have syntax errors before you even run a program. Once you've used a modern editor, you'll wonder how you ever managed without one.

There are a number of good programs available, but I have settled on Visual Studio Code (VSC) from Microsoft because it's powerful; runs on all of Windows, Mac, and Linux; and is free (see Figure 2-12). Everyone has different programming styles and preferences, though, so if you don't get on with it, there are plenty more code editors available to choose from—or you may wish to go directly for an integrated development environment (IDE), as described in the following section.

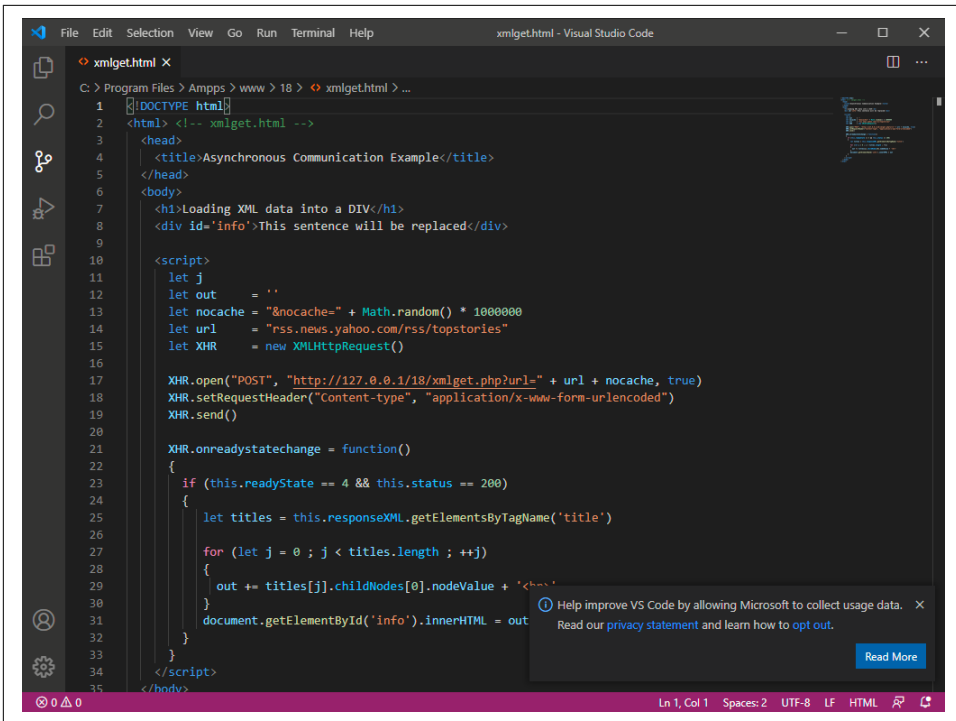


Figure 2-12. Program editors (like Visual Studio Code) are superior to plain-text editors

As you can see from **Figure 2-12**, VSC highlights the syntax appropriately, using colors to help clarify what's going on. What's more, you can place the cursor next to brackets or braces, and it will highlight the matching ones so that you can check whether you have too many or too few. In fact, VSC does a lot more in addition, which you will discover and enjoy as you use it. You can download a copy from code.visualstudio.com.

Again, if you have a different preferred program editor, use that; it's always a good idea to use programs you're already familiar with.

Using an IDE

As good as dedicated program editors can be for your programming productivity, their utility pales when compared to integrated development environments. IDEs offer many additional features such as in-editor debugging and program testing, as well as function descriptions and much more, although these are features that are slowly creeping into some code editors too these days, such as the previously recommended VSC. **Figure 2-13** shows the popular Eclipse IDE with some HTML loaded

into the main frame (it allows you to work with PHP, JavaScript, and other file types as well).

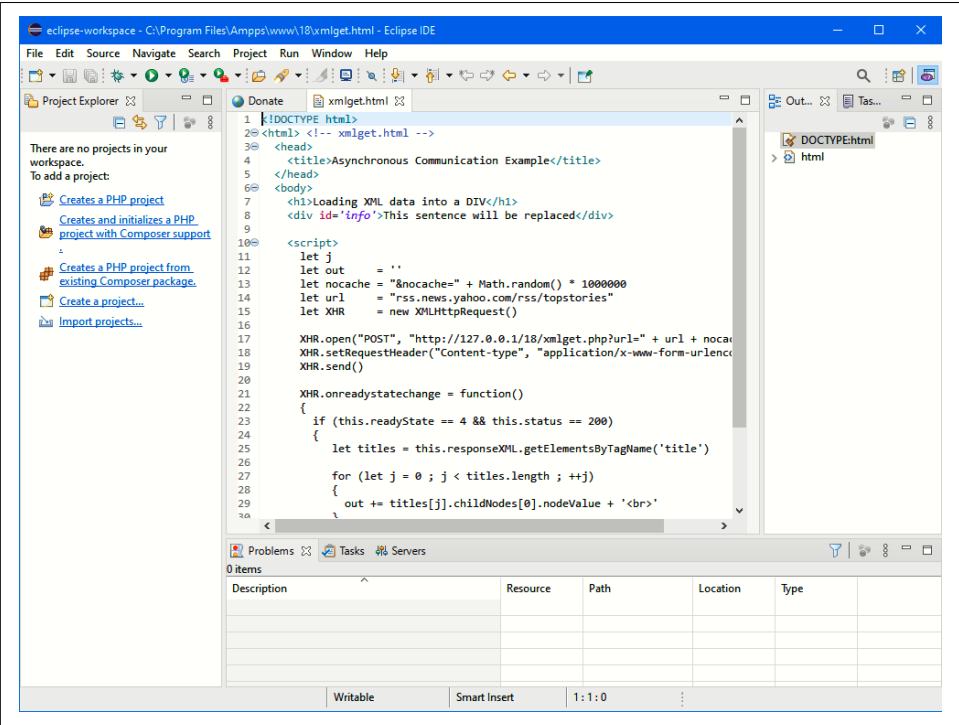


Figure 2-13. When you’re using an IDE, code development becomes much quicker and easier

When developing with an IDE, you can set breakpoints and then run all (or portions) of your code, which will stop at the breakpoints and provide you with information about the program’s current state.

As an aid to learning programming, the examples in this book can be entered into an IDE and run there and then, without the need to call up your web browser. There are several IDEs available for different platforms. Table 2-1 lists some of the most popular free PHP IDEs, along with their download URLs.

Table 2-1. A selection of free IDEs

IDE	Download URL	Windows	macOS	Linux
Eclipse PDT	eclipse.org/downloads/packages	✓	✓	✓
NetBeans	www.netbeans.org	✓	✓	✓
Visual Studio	code.visualstudio.com	✓	✓	✓

Choosing an IDE can be a very personal thing, so if you intend to use one, I advise you to download a couple or more to try them out first; they all either have trial versions or are free to use, so it won't cost you anything.

You should take the time to install a code editor or IDE you are comfortable with now; you'll then be ready to try out the examples in the coming chapters.

Armed with these tools, you are now ready to move on to [Chapter 3](#), where we'll start exploring PHP in further depth and find out how to get HTML and PHP to work together, as well as how the PHP language itself is structured. But before moving on, I suggest you test your new knowledge with the following questions.

Questions

1. What is the difference between a WAMP, a MAMP, and a LAMP?
2. What do the IP address 127.0.0.1 and the URL *http://localhost* have in common?
3. What is the purpose of an FTP program?
4. Name the main disadvantage of working on a remote web server.
5. Why is it better to use a program editor instead of a plain-text editor?

See [“Chapter 2 Answers” on page 748](#) in the [Appendix](#) for the answers to these questions.

Introduction to PHP

In [Chapter 1](#), I explained that PHP is the language that you use to make the server generate dynamic output—output that is potentially different each time a browser requests a page. In this chapter, you’ll start learning this simple but powerful language; it will be the topic of the following chapters up through [Chapter 7](#).

I encourage you to develop your PHP code using one of the IDEs listed in [Chapter 2](#), or a good code editor.

Many of these programs will let you run the PHP code and see the output discussed in this chapter. I’ll also show you how to create PHP code so that you can see what the output looks like in a web page (the way your users will ultimately see it). But that step, as thrilling as it may be at first, isn’t really important at this stage.

In production, your web pages will be a combination of PHP, HTML, JavaScript, and some MySQL statements laid out using CSS. Furthermore, each page can lead to other pages to provide users with ways to click through links and fill out forms. We can avoid all that complexity while learning each language, though. Focus for now on just writing PHP code and making sure that you get the output you expect—or at least that you understand the output you actually get!

Incorporating PHP Within HTML

By default, PHP documents end with the extension *.php*. When a web server encounters this extension in a requested file, it automatically passes it to the PHP processor. Of course, web servers are highly configurable, and some web developers choose to force files ending with *.htm* or *.html* to also get parsed by the PHP processor, usually because they want to hide their use of PHP.

Your PHP program is responsible for passing back a clean file suitable for display in a web browser. At its very simplest, a PHP document will output only HTML. To prove this, you can take any normal HTML document and save it as a PHP document (for example, saving *index.html* as *index.php*), and it will display identically to the original.

To trigger the PHP commands, you need to learn a new tag. Here is the first part:

```
<?php
```

The first thing you may notice is that the tag has not been closed. This is because entire sections of PHP can be placed inside this tag, and they finish only when the closing part is encountered, which looks like this:

```
?>
```

A small PHP “Hello World” program might look like [Example 3-1](#).

Example 3-1. Invoking PHP

```
<?php
echo "Hello world";
?>
```

Use of this tag can be quite flexible. Some programmers open the tag at the start of a document and close it right at the end, outputting any HTML directly from PHP commands. Others, however, choose to insert only the smallest possible fragments of PHP within these tags wherever dynamic scripting is required, leaving the rest of the document in standard HTML.

The latter type of programmer generally argues that their style of coding results in faster code, while the former says that the speed increase is so minimal that it doesn’t justify the additional complexity of dropping in and out of PHP many times in a single document.

As you learn more, you will surely discover your preferred style of PHP development, but for the sake of making the examples in this book easier to follow, I have adopted the approach of keeping the number of transfers between PHP and HTML to a minimum—generally only once or twice in a document.

By the way, there is a slight variation to the PHP syntax. If you browse the internet for PHP examples, you may also encounter code where the opening and closing syntax looks like this:

```
<?
echo "Hello world";
?>
```

Although it’s not as obvious that the PHP parser is being called, this is a valid, alternative syntax that also usually works. But I discourage its use, as it is incompatible

with XML and is now deprecated (meaning that it is no longer recommended and support could be removed in future versions).



If you have only PHP code in a file, you may omit the closing `?>`. This can be a good practice, as it will ensure that you have no excess whitespace leaking from your PHP files (especially important when you're writing object-oriented code).

This Book's Examples

To save you the time it would take to type them all in, all the examples from this book have been stored at GitHub. You can download the archive to your computer by visiting: [GitHub](#).

In addition to listing all the examples by chapter and example number (such as *example3-1.php*), some of the examples may require explicit filenames, in which case copies of the example(s) are also saved using the filename(s) in the same folder (such as the upcoming [Example 3-4](#), which should be saved as *test1.php*).

The Structure of PHP

We're going to cover quite a lot of ground in this section. It's not too difficult, but I recommend that you work your way through it carefully, as it sets the foundation for everything else in this book. As always, there are some useful questions at the end of the chapter that you can use to test how much you've learned.

Using Comments

There are two ways in which you can add comments to your PHP code. The first turns a single line into a comment by preceding it with a pair of forward slashes:

```
// This is a comment
```

This version of the comment feature is a great way to temporarily remove a line of code from a program that is giving you errors. For example, you could use such a comment to hide a debugging line of code until you need it, like this:

```
// echo "X equals $x";
```

You can also use this type of comment directly after a line of code to describe its action, like this:

```
$x += 10; // Increment $x by 10
```

When you need to use multiple lines, there's a second type of comment, which looks like [Example 3-2](#).

Example 3-2. A multiline comment

```
<?php
/* This is a section
   of multiline comments
   which will not be
   interpreted */
?>
```

You can use the `/*` and `*/` pairs of characters to open and close comments almost anywhere you like inside your code. Most, if not all, programmers use this construct to temporarily comment out entire sections of code that do not work or that, for one reason or another, they do not wish to be interpreted.



A common error is to use `/*` and `*/` to comment out a large section of code that already contains a commented-out section that uses those characters. You can't nest comments this way; the PHP interpreter won't know where a comment ends and will display an error message. However, if you use an editor or IDE with syntax highlighting, this type of error is easier to spot.

Basic Syntax

PHP is quite a simple language with roots in C and Perl (if you have ever come across these), yet it looks more like Java. It is also very flexible, but there are a few rules that you need to learn about its syntax and structure.

Semicolons

You may have noticed in the previous examples that the PHP commands ended with a semicolon, like this:

```
$x += 10;
```

One of the most common causes of errors you will encounter with PHP is forgetting this semicolon. This causes PHP to treat multiple statements like one statement, which it is unable to understand, prompting it to produce a `Parse error` message.

The \$ symbol

The `$` symbol has come to be used in many different ways by different programming languages. For example, in the BASIC language, it was used to terminate variable names to denote them as strings.

In PHP, however, you must place a `$` in front of *all* variables. This is required to make the PHP parser faster, as it instantly knows whenever it comes across a variable.

Whether your variables are numbers, strings, or arrays, they should all look something like those in [Example 3-3](#).

Example 3-3. Three different types of variable assignment

```
<?php
$mycounter = 1;
$string    = "Hello";
$array     = array("One", "Two", "Three");
?>
```

And really that's pretty much all the syntax that you have to remember. Unlike languages such as Python, which are very strict about how you indent and lay out your code, PHP leaves you completely free to use (or not use) all the indenting and spacing you like. In fact, sensible use of whitespace is generally encouraged (along with comprehensive commenting) to help you understand your code when you come back to it. It also helps other programmers when they have to maintain your code.

Variables

There's a simple metaphor that will help you understand what PHP variables are all about. Just think of them as little (or big) matchboxes! That's right—matchboxes that you've painted over and written names on.

String variables

Imagine you have a matchbox on which you have written the word *username*. You then write *Fred Smith* on a piece of paper and place it into the box (see [Figure 3-1](#)). Well, that's the same process as assigning a string value to a variable, like this:

```
$username = "Fred Smith";
```

The quotation marks indicate that “Fred Smith” is a *string* of characters. You must enclose each string in either quotation marks or apostrophes (single quotes), although there is a subtle difference between the two types of quote, which is explained later. When you want to see what's in the box, you open it, take the piece of paper out, and read it. In PHP, doing so looks like this (which displays the contents of the variable):

```
echo $username;
```

Or you can assign it to another variable (photocopy the paper and place the copy in another matchbox), like this:

```
$current_user = $username;
```

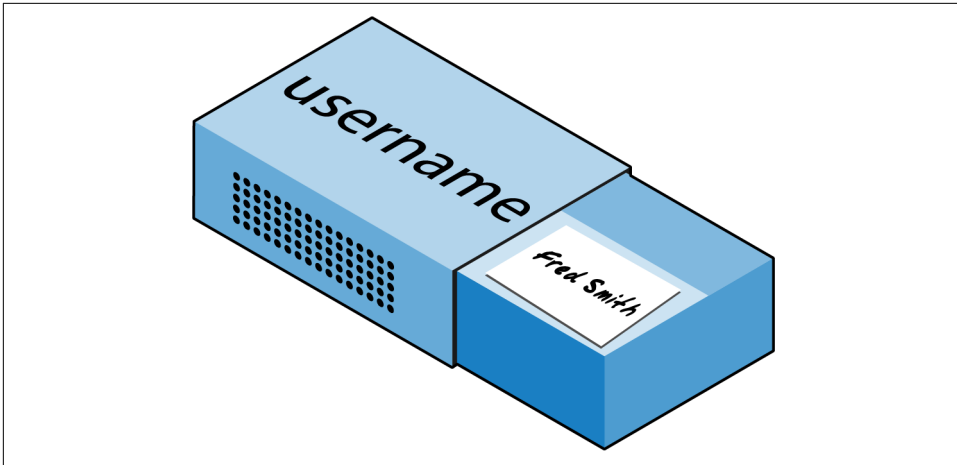


Figure 3-1. You can think of variables as matchboxes containing items

Example 3-4. Your first PHP program

```
<?php // test1.php
$username = "Fred Smith";
echo $username;
echo "<br>";
$current_user = $username;
echo $current_user;
?>
```

Now you can call it up by entering the following into your browser's address bar:

`http://localhost/test1.php`



In the unlikely event that during the installation of your web server (as detailed in [Chapter 2](#)) you changed the port assigned to the server to anything other than 80, then you must place that port number within the URL in this and all other examples in this book. So, for example, if you changed the port to 8080, the preceding URL would become this:

`http://localhost:8080/test1.php`

I won't mention this again, so just remember to use the port number (if required) when trying examples or writing your own code.

The result of running this code should be two occurrences of the name *Fred Smith*, the first of which is the result of the `echo $username` command and the second of which is the result of the `echo $current_user` command.

Numeric variables

Variables don't have to contain just strings—they can contain numbers too. If we return to the matchbox analogy, to store the number 17 in the variable `$count`, the equivalent would be placing, say, 17 beads in a matchbox on which you have written the word *count*:

```
$count = 17;
```

You could also use a floating-point number (containing a decimal point). The syntax is the same:

```
$count = 17.5;
```

To see the contents of the matchbox, you would simply open it and count the beads. In PHP, you would assign the value of `$count` to another variable or perhaps just echo it to the web browser.

Arrays

You can think of arrays as several matchboxes glued together. For example, let's say we want to store the player names for a five-person soccer team in an array called `$team`. To do this, we could glue five matchboxes side by side and write down the names of all the players on separate pieces of paper, placing one in each matchbox.

Across the top of the whole matchbox assembly we would write the word *team* (see [Figure 3-2](#)). The equivalent of this in PHP would be the following:

```
$team = array('Bill', 'Mary', 'Mike', 'Chris', 'Anne');
```

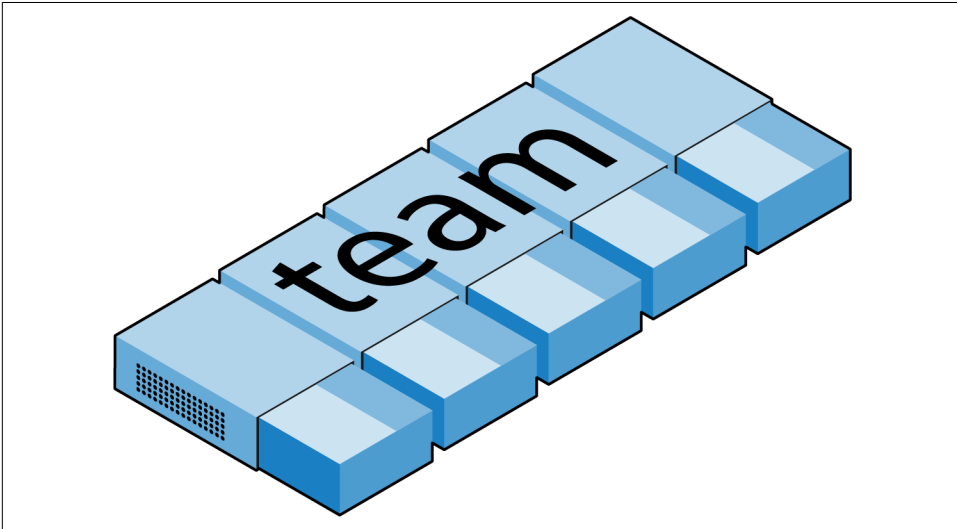


Figure 3-2. An array is like several matchboxes glued together

This syntax is more complicated than the other examples you've seen so far. The array-building code consists of the following construct:

```
array();
```

with five strings inside. Each string is enclosed in apostrophes or quotes, and strings must be separated with commas.

If we then wanted to know who player 4 is, we could use this command:

```
echo $team[3]; // Displays the name Chris
```

The reason the previous statement has the number 3, not 4, is that the first element of a PHP array is actually the zeroth element, so the player numbers will therefore be 0 through 4.

Two-dimensional arrays

There's a lot more you can do with arrays. For example, instead of being single-dimensional lines of matchboxes, they can be two-dimensional matrixes or even have more dimensions.

As an example of a two-dimensional array, let's say we want to keep track of a game of tic-tac-toe, which requires a data structure of nine cells arranged in a 3×3 square. To represent this with matchboxes, imagine nine of them glued to one other in a matrix of three rows by three columns (see [Figure 3-3](#)).

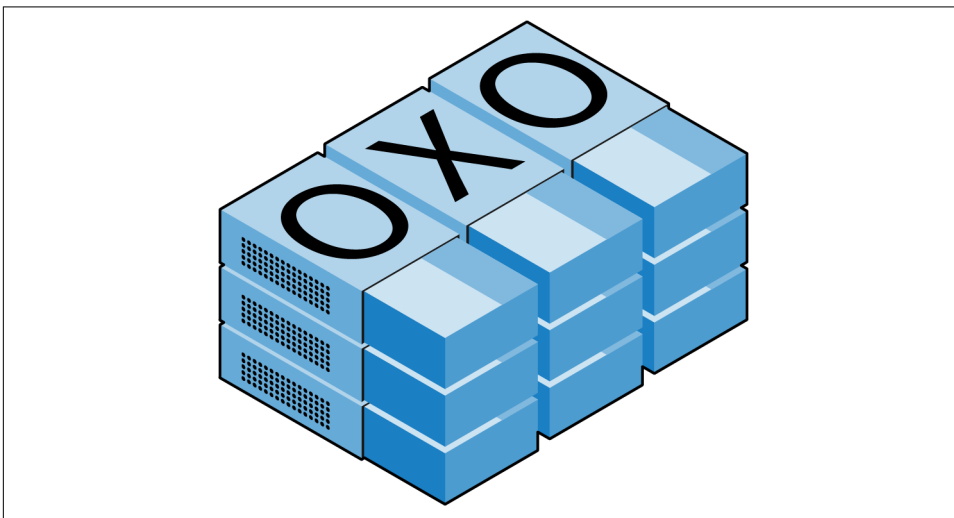


Figure 3-3. A multidimensional array simulated with matchboxes

You can now place a piece of paper with either an *x* or an *o* on it in the correct matchbox for each move played. To do this in PHP code, you have to set up an array con-

taining three more arrays, as in [Example 3-5](#), in which the array is set up with a game already in progress.

Example 3-5. Defining a two-dimensional array

```
<?php
$oxo = array(array('x', ' ', 'o'),
              array('o', 'o', 'x'),
              array('x', 'o', ' '));
?>
```

Once again, we've moved up a step in complexity, but it's easy to understand if you grasp the basic array syntax. There are three `array()` constructs nested inside the outer `array()` construct. We've filled each row with an array consisting of just one character: an *x*, an *o*, or a blank space. (We use a blank space so that all the cells will be the same width when they are displayed.)

To then return the third element in the second row of this array, you would use the following PHP command, which will display an *x*:

```
echo $oxo[1][2];
```



Remember that array indexes (pointers at elements within an array) start from zero, not one, so the `[1]` in the previous command refers to the second of the three arrays, and the `[2]` references the third position within that array. This command will return the contents of the matchbox three along and two down.

As mentioned, we can support arrays with even more dimensions by simply creating more arrays within arrays. However, we will not be covering arrays of more than two dimensions in this book.

And don't worry if you're still having difficulty coming to grips with using arrays, as the subject is explained in detail in [Chapter 6](#).

Variable-naming rules

When creating PHP variables, you must follow these four rules:

- Variable names, after the dollar sign, must start with a letter of the alphabet or the `_` (underscore) character.
- Variable names can contain only the characters `a-z`, `A-Z`, `0-9`, and `_` (underscore).

- Variable names may not contain spaces. If a variable name must comprise more than one word, a good idea is to separate the words with the `_` (underscore) character (e.g., `$user_name`).
- Variable names are case-sensitive. The variable `$High_Score` is not the same as the variable `$high_score`.



To allow extended ASCII characters that include accents, PHP also supports the bytes from 127 through 255 in variable names. But unless your code will be maintained only by programmers who are used to those characters, it's probably best to avoid them, because programmers using English keyboards will have difficulty accessing them.

Operators

Operators let you specify mathematical operations to perform, such as addition, subtraction, multiplication, and division. But several other types of operators exist too, such as the string, comparison, and logical operators. Math in PHP looks a lot like plain arithmetic—for instance, the following statement outputs 8:

```
echo 6 + 2;
```

Before moving on to learn what PHP can do for you, take a moment to learn about the various operators it provides.

Arithmetic operators

Arithmetic operators do what you would expect—they are used to perform mathematics. You can use them for the main four operations (add, subtract, multiply, and divide) as well as to find a modulus (the remainder after a division) and to increment or decrement a value (see [Table 3-1](#)).

Table 3-1. Arithmetic operators

Operator	Description	Example
+	Addition	<code>\$j + 1</code>
-	Subtraction	<code>\$j - 6</code>
*	Multiplication	<code>\$j * 11</code>
/	Division	<code>\$j / 4</code>
%	Modulus (the remainder after a division is performed)	<code>\$j % 9</code>
++	Increment	<code>++\$j</code>
--	Decrement	<code>--\$j</code>
**	Exponentiation (or power)	<code>\$j**2</code>

Assignment operators

These operators assign values to variables. They start with the very simple = and move on to +=, -=, and so on (see [Table 3-2](#)). The operator += adds the value on the right side to the variable on the left, instead of totally replacing the value on the left. Thus, if \$count starts with the value 5, the statement:

```
$count += 1;
```

sets \$count to 6, just like the more familiar assignment statement:

```
$count = $count + 1;
```

The /= and *= operators are similar, but for division and multiplication, the .= operator concatenates variables, such that \$a .= "." will append a period to the end of \$a, and %= assigns a percentage value.

Table 3-2. Assignment operators

Operator	Example	Equivalent to
=	<code>\$j = 15</code>	<code>\$j = 15</code>
+=	<code>\$j += 5</code>	<code>\$j = \$j + 5</code>
-=	<code>\$j -= 3</code>	<code>\$j = \$j - 3</code>
*=	<code>\$j *= 8</code>	<code>\$j = \$j * 8</code>
/=	<code>\$j /= 16</code>	<code>\$j = \$j / 16</code>
.=	<code>\$j .= \$k</code>	<code>\$j = \$j . \$k</code>
%=	<code>\$j %= 4</code>	<code>\$j = \$j % 4</code>

Comparison operators

Comparison operators are generally used inside a construct such as an `if` statement in which you need to compare two items. For example, you may wish to know whether a variable you have been incrementing has reached a specific value, or whether another variable is less than a set value, and so on (see [Table 3-3](#)).

Table 3-3. Comparison operators

Operator	Description	Example
<code>==</code>	Is equal to	<code>\$j == 4</code>
<code>!=</code>	Is not equal to	<code>\$j != 21</code>
<code>></code>	Is greater than	<code>\$j > 3</code>
<code><</code>	Is less than	<code>\$j < 100</code>
<code>>=</code>	Is greater than or equal to	<code>\$j >= 15</code>
<code><=</code>	Is less than or equal to	<code>\$j <= 8</code>
<code><></code>	Is not equal to	<code>\$j <> 23</code>
<code>===</code>	Is identical to	<code>\$j === "987"</code>
<code>!==</code>	Is not identical to	<code>\$j !== "1.2e3"</code>

Note the difference between `=` and `==`. The first is an assignment operator, and the second is a comparison operator. Even advanced programmers can sometimes mix up the two when coding hurriedly, so be careful.

Logical operators

If you haven't used them before, logical operators may at first seem a little daunting. But just think of them the way you would use logic in English. For example, you might say to yourself, "If the time is later than 12 p.m. and earlier than 2 p.m., have lunch." In PHP, the code for this might look something like the following (using military time):

```
if ($hour > 12 && $hour < 14) doLunch();
```

Here we have moved the set of instructions for actually going to lunch into a function that we will have to create later called `doLunch`.

As the previous example shows, you generally use a logical operator to combine the results of two of the comparison operators shown in the previous section. A logical operator can also be input to another logical operator: “If the time is later than 12 p.m. and earlier than 2 p.m., or if the smell of a roast is permeating the hallway and there are plates on the table.” As a rule, if something has a TRUE or FALSE value, it can be input to a logical operator. A logical operator takes two true or false inputs and produces a true or false result.

Table 3-4 shows the logical operators.

Table 3-4. Logical operators

Operator	Description	Example
<code>&&</code>	<i>And</i>	<code>\$j == 3 && \$k == 2</code>
<code>and</code>	Low-precedence <i>and</i>	<code>\$j == 3 and \$k == 2</code>
<code> </code>	<i>Or</i>	<code>\$j < 5 \$j > 10</code>
<code>or</code>	Low-precedence <i>or</i>	<code>\$j < 5 or \$j > 10</code>
<code>!</code>	<i>Not</i>	<code>! (\$j == \$k)</code>
<code>xor</code>	<i>Exclusive or</i>	<code>\$j xor \$k</code>

Note that `&&` is usually interchangeable with `and`; the same is true for `||` and `or`. However, because `and` and `or` have a lower precedence, you should avoid using them except when they are the only option, as in the following statement, which *must* use the `or` operator (`||` cannot be used to force a second statement to execute if the first fails):

```
$html = file_get_contents($site) or die("Cannot download from $site");
```

The most unusual of these operators is `xor`, which stands for *exclusive or* and returns a TRUE value if either value is TRUE but a FALSE value if both inputs are TRUE or both inputs are FALSE. To understand this, imagine that you want to concoct your own cleaner for household items. Ammonia makes a good cleaner, and so does bleach, so you want your cleaner to have one of these. But the cleaner must not have both, because the combination is hazardous. In PHP, you could represent this as follows:

```
$ingredient = $ammonia xor $bleach;
```

In this example, if either `$ammonia` or `$bleach` is TRUE, `$ingredient` will also be set to TRUE. But if both are TRUE or both are FALSE, `$ingredient` will be set to FALSE.

Variable Assignment

The syntax to assign a value to a variable is always *variable = value*. Or, to reassign the value to another variable, it is *other_variable = variable*.

There are also a couple of other assignment operators that you will find useful. For example, we've already seen this:

```
$x += 10;
```

which tells the PHP parser to add the value on the right (in this instance, the value 10) to the variable \$x. Likewise, we could subtract as follows:

```
$y -= 10;
```

Variable incrementing and decrementing

Adding or subtracting 1 is such a common operation that PHP provides special operators for it. You can use one of the following in place of the += and -= operators:

```
++$x;  
--$y;
```

In conjunction with a test (an if statement), you could use the following code:

```
if (++$x == 10) echo $x;
```

This tells PHP to *first* increment the value of \$x and then to test whether it has the value 10 and, if it does, to output its value. But you can also require PHP to increment (or, as in the following example, decrement) a variable *after* it has tested the value, like this:

```
if ($y-- == 0) echo $y;
```

which gives a subtly different result. Suppose \$y starts out as 0 before the statement is executed. The comparison will return a TRUE result, but \$y will be set to -1 after the comparison is made. So what will the echo statement display: 0 or -1? Try to guess, and then try out the statement in a PHP processor to confirm. Because this combination of statements is confusing, it should be taken as just an educational example and not as a guide to good programming style.

In short, a variable is incremented or decremented before the test if the operator is placed before the variable, whereas the variable is incremented or decremented after the test if the operator is placed after the variable.

By the way, the correct answer to the previous question is that the echo statement will display the result -1, because \$y was decremented right after it was accessed in the if statement, and before the echo statement.

String concatenation

Concatenation is a somewhat arcane term for putting something after another thing. So, string concatenation uses the period (.) to append one string of characters to another. The simplest way to do this is as follows:

```
echo "You have " . $msgs . " messages.";
```

Assuming that the variable `$msgs` is set to the value 5, the output from this line of code will be the following:

```
You have 5 messages.
```

Just as you can add a value to a numeric variable with the `+=` operator, you can append one string to another using `.=`, like this:

```
$bulletin .= $newsflash;
```

In this case, if `$bulletin` contains a news bulletin and `$newsflash` has a news flash, the command appends the news flash to the news bulletin so that `$bulletin` now comprises both strings of text.

String types

PHP supports two types of strings that are denoted by the type of quotation mark that you use. If you wish to assign a literal string, preserving the exact contents, you should use single quotation marks (apostrophes), like this:

```
$info = 'Preface variables with a $ like this: $variable';
```

In this case, every character within the single-quoted string is assigned to `$info`. If you had used double quotes, PHP would have attempted to evaluate `$variable` as a variable.

On the other hand, when you want to include the value of a variable inside a string, you do so by using double-quoted strings:

```
echo "This week $count people have viewed your profile";
```

As you will realize, this syntax also offers a simpler option to concatenation in which you don't need to use a period, or close and reopen quotes, to append one string to another. This is called *variable substitution*, and some programmers use it extensively, whereas others don't use it at all.

Escaping characters

Sometimes a string needs to contain characters with special meanings that might be interpreted incorrectly. For example, the following line of code will not work, because the second quotation mark encountered in the word *spelling's* will tell the PHP parser that the string's end has been reached. Consequently, the rest of the line will be rejected as an error:

```
$text = 'My spelling's atrocious'; // Erroneous syntax
```

To correct this, you can add a backslash directly before the offending quotation mark to tell PHP to treat the character literally and not to interpret it:

```
$text = 'My spelling\'s still atrocious';
```

And you can perform this trick in almost all situations in which PHP would otherwise return an error by trying to interpret a character. For example, the following double-quoted string will be correctly assigned:

```
$text = "She wrote upon it, \"Return to sender\".";
```

Additionally, you can use escape characters to insert various special characters into strings, such as tabs, newlines, and carriage returns. These are represented, as you might guess, by `\t`, `\n`, and `\r`. Here is an example using tabs to lay out a heading—it is included here merely to illustrate escapes, because in web pages there are always better ways to do layout:

```
$heading = "Date\tName\tPayment";
```

These special backslash-preceded characters work only in double-quoted strings. In single-quoted strings, the preceding string would be displayed with the ugly `\t` sequences instead of tabs. Within single-quoted strings, only the escaped apostrophe (`\'`) and escaped backslash itself (`\\`) are recognized as escaped characters.

Multiline Commands

There are times when you need to output quite a lot of text from PHP, and using several `echo` (or `print`) statements would be time-consuming and messy. To overcome this, PHP offers two conveniences. The first is just to put multiple lines between quotes, as in [Example 3-6](#). Variables can also be assigned, as in [Example 3-7](#).

Example 3-6. A multiline string `echo` statement

```
<?php
    $author = "Steve Ballmer";

    echo "Developers, developers, developers, developers, developers,
    developers, developers, developers, developers!

    - $author.";
?>
```

Example 3-7. A multiline string assignment

```
<?php
    $author = "Bill Gates";
```



```
$text = "Measuring programming progress by lines of code is like
Measuring aircraft building progress by weight.

- $author.";
?>
```

PHP also offers a multiline sequence using the `<<<` operator—commonly referred to as a *here-document* or *heredoc*—as a way of specifying a string literal, preserving the line breaks and other whitespace (including indentation) in the text. Its use can be seen in [Example 3-8](#).

Example 3-8. Alternative multiline echo statement

```
<?php
$author = "Brian W. Kernighan";

echo <<<_END
Debugging is twice as hard as writing the code in the first place.
Therefore, if you write the code as cleverly as possible, you are,
by definition, not smart enough to debug it.

- $author.
_END;
?>
```

This code tells PHP to output everything between the two `_END` tags as if it were a double-quoted string (except that quotes in a heredoc do not need to be escaped). This means it's possible, for example, for a developer to write entire sections of HTML directly into PHP code and then just replace specific dynamic parts with PHP variables.

It is important to remember that the closing `_END`; *must* appear right at the start of a new line, and it must be the *only* thing on that line—not even a comment is allowed to be added after it (nor even a single space). Once you have closed a multiline block, you are free to use the same tag name again.



Remember: using the `<<<_END..._END`; heredoc construct, you don't have to add `\n` linefeed characters to send a linefeed—just press Return and start a new line. Also, unlike in either a double-quote- or single-quote-delimited string, you are free to use all the single and double quotes you like within a heredoc, without escaping them by preceding them with a backslash (`\`).

[Example 3-9](#) shows how to use the same syntax to assign multiple lines to a variable.

Example 3-9. A multiline string variable assignment

```
<?php
    $author = "Scott Adams";

    $out = <<<_END
    Normal people believe that if it ain't broke, don't fix it.
    Engineers believe that if it ain't broke, it doesn't have enough
    features yet.

    - $author.
    _END;
echo $out;
?>
```

The variable `$out` will then be populated with the contents between the two tags. If you were appending, rather than assigning, you could also have used `.=` in place of `=` to append the string to `$out`.

Be careful not to place a semicolon directly after the first occurrence of `_END`, as that would terminate the multiline block before it had even started and cause a Parse error message.

By the way, the `_END` tag is simply one I chose for these examples because it is unlikely to be used anywhere else in PHP code and is therefore unique. You can use any tag you like, such as `_SECTION1` or `_OUTPUT` and so on. Also, to help differentiate tags such as this from variables or functions, the general practice is to preface them with an underscore.



Laying out text over multiple lines is usually just a convenience to make your PHP code easier to read, because once it is displayed in a web page, HTML formatting rules take over and whitespace is suppressed (but `$author` in our example will still be replaced with the variable's value).

So, for example, if you load these multiline output examples into a browser, they will *not* display over several lines, because all browsers treat newlines just like spaces. However, if you use the browser's View Source feature, you will find that the newlines are correctly placed and that PHP preserved the line breaks.

Variable Typing

PHP is a loosely typed language. This means that variables do not have to be declared before they are used and that PHP always converts variables to the type required by their context when they are accessed.

For example, you can create a multiple-digit number and extract the n th digit from it simply by assuming it to be a string. In [Example 3-10](#), the numbers 12345 and 67890 are multiplied together, returning a result of 838102050, which is then placed in the variable `$number`.

Example 3-10. Automatic conversion from a number to a string

```
<?php
    $number = 12345 * 67890;
    echo substr($number, 3, 1);
?>
```

At the point of the assignment, `$number` is a numeric variable. But on the second line, a call is placed to the PHP function `substr`, which asks for one character to be returned from `$number`, starting at the fourth position (remember that PHP offsets start from zero). To do this, PHP turns `$number` into a nine-character string so that `substr` can access it and return the character, which in this case is 1.

The same goes for turning a string into a number, and so on. In [Example 3-11](#), the variable `$pi` is set to a string value, which is then automatically turned into a floating-point number in the third line by the equation for calculating a circle's area, which outputs the value 78.5398175.

Example 3-11. Automatically converting a string to a number

```
<?php
    $pi      = "3.1415927";
    $radius = 5;
    echo $pi * ($radius * $radius);
?>
```

In practice, what this all means is that you don't have to worry too much about your variable types. Just assign them values that make sense to you, and PHP will convert them if necessary. Then, when you want to retrieve values, just ask for them—for example, with an `echo` statement, but do remember that sometimes automatic conversions do not operate quite as you might expect.

Constants

Constants are similar to variables, holding information to be accessed later, except that they are what they sound like—constant. In other words, once you have defined one, its value is set for the remainder of the program and cannot be altered.

For example, you can use a constant to hold the location of your server root (the folder with the main files of your website). You would define such a constant like this: